

AMA 564 Deep Learning

2026 Spring

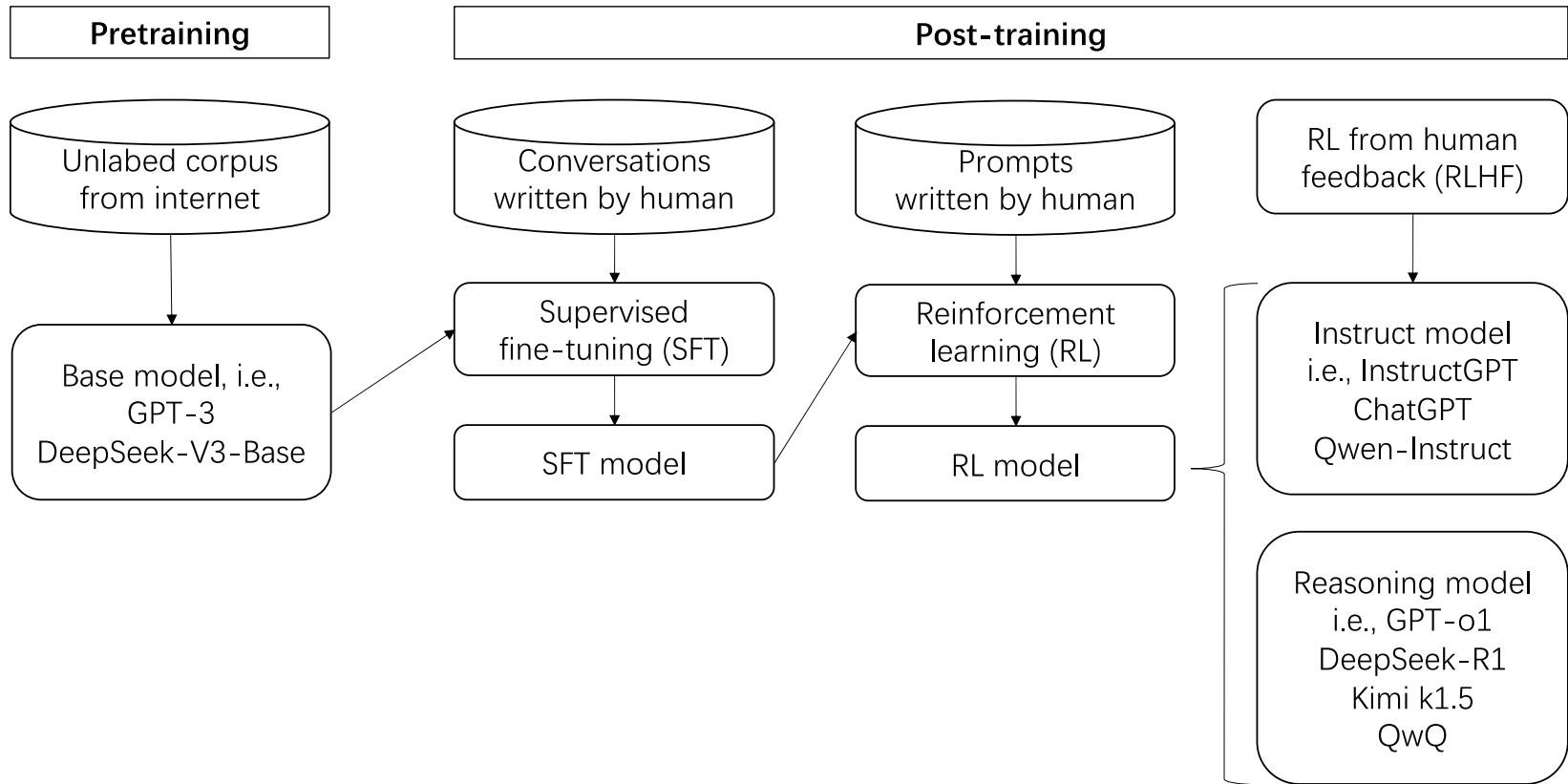
Lecture 11



- **Pretraining**
- **BERT**
- **PEFT**

Part of the contents are inspired and adopted from Stanford CS224n.

An overview of LLM training



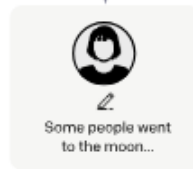
Step 1

Collect demonstration data, and train a supervised policy.

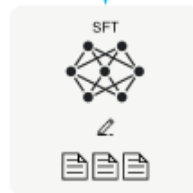
A prompt is sampled from our prompt dataset.



A labeler demonstrates the desired output behavior.



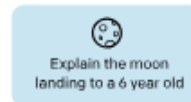
This data is used to fine-tune GPT-3 with supervised learning.



Step 2

Collect comparison data, and train a reward model.

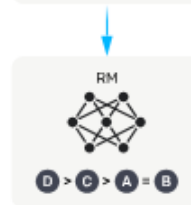
A prompt and several model outputs are sampled.



A labeler ranks the outputs from best to worst.



This data is used to train our reward model.



Step 3

Optimize a policy against the reward model using reinforcement learning.

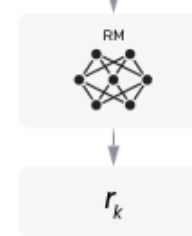
A new prompt is sampled from the dataset.



The policy generates an output.



The reward model calculates a reward for the output.



The reward is used to update the policy using PPO.

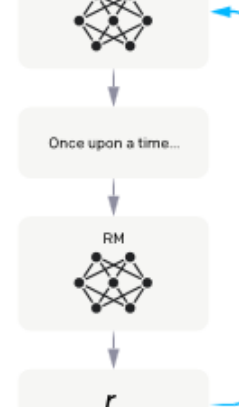
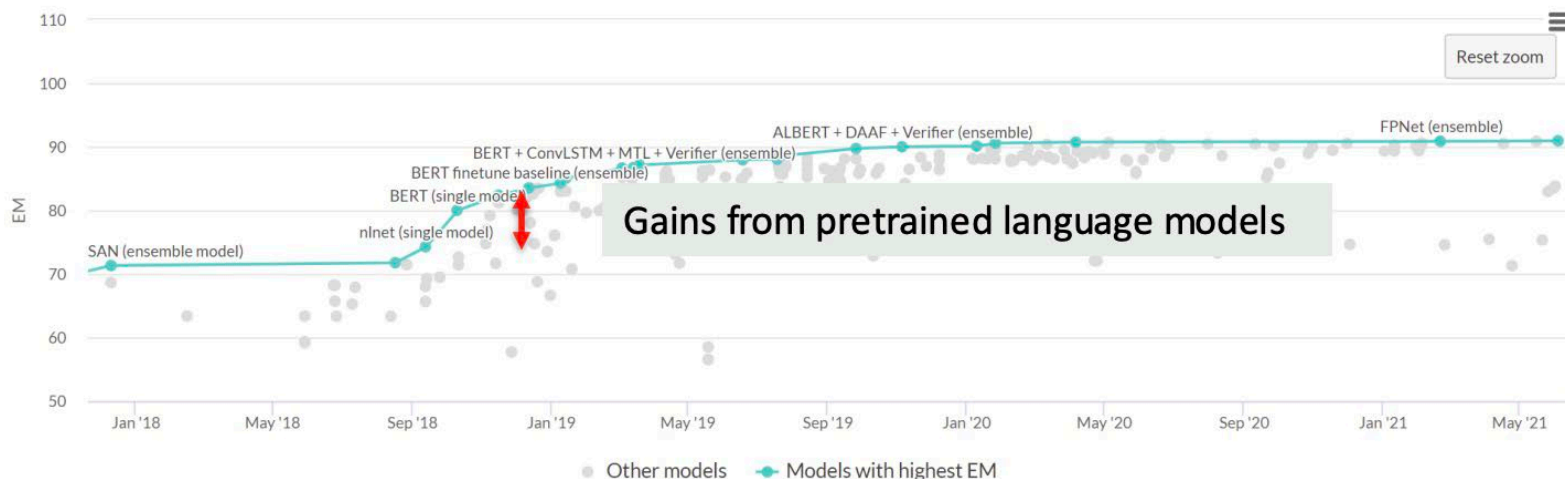


Figure 2: A diagram illustrating the three steps of our method: (1) supervised fine-tuning (SFT), (2) reward model (RM) training, and (3) reinforcement learning via proximal policy optimization (PPO) on this reward model. Blue arrows indicate that this data is used to train one of our models. In Step 2, boxes A-D are samples from our models that get ranked by labelers. See Section 3 for more details on our method.

Pretraining

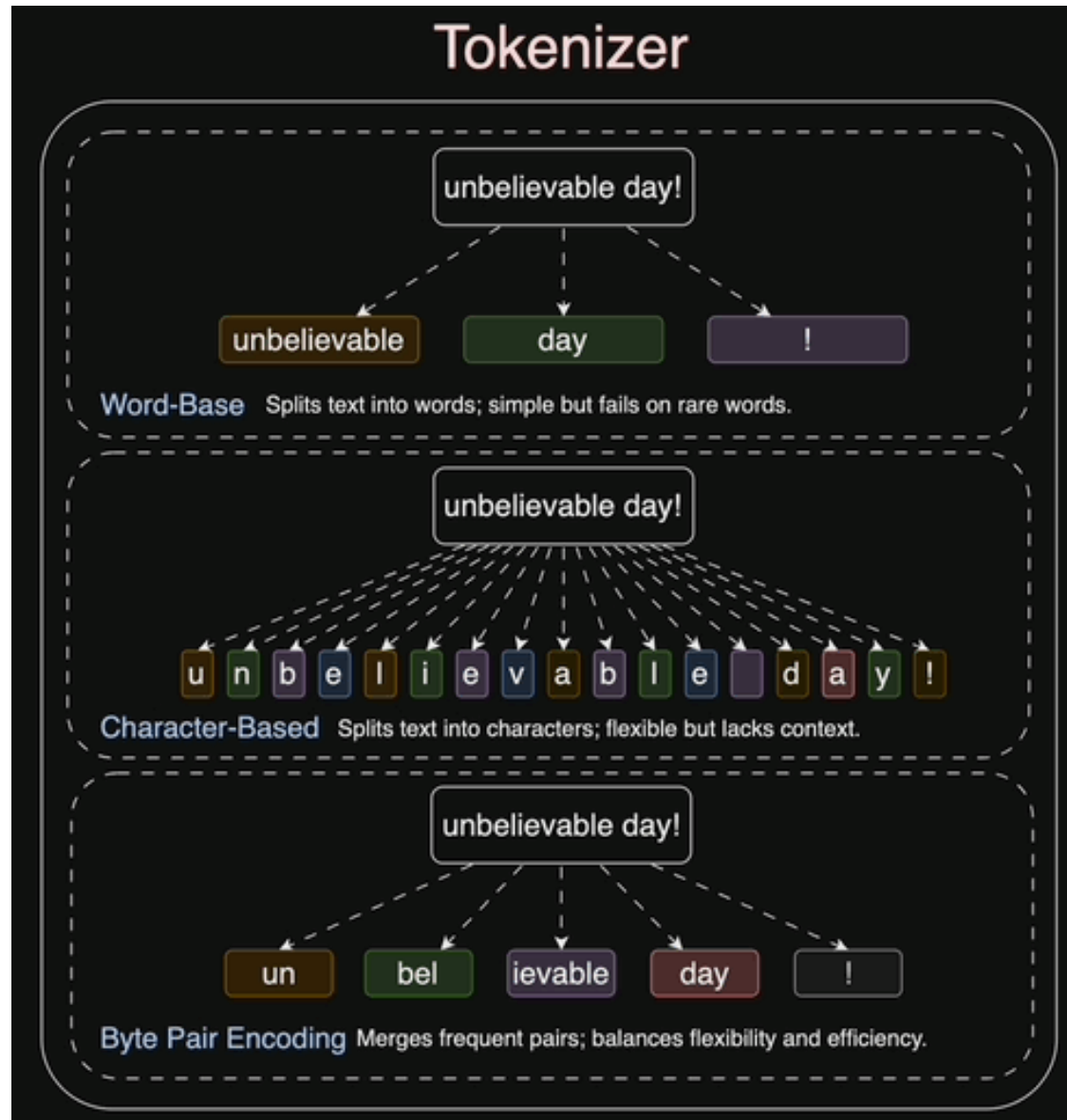
The Pretraining Revolution



Pretraining has had a major, tangible impact on how well NLP systems work

Keys in pretraining:

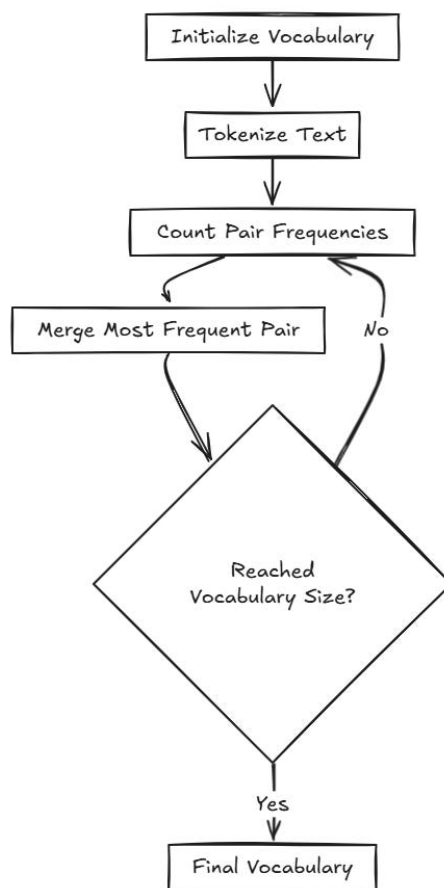
1. Make sure your model can process large-scale, diverse datasets.
2. Use unlabeled data.
3. Compute-aware scaling.



The Byte-pair Encoding Algorithm



“Byte Pair Encoding (BPE) is a data compression technique that iteratively merges the most frequent pair of consecutive bytes (or characters) in a text or data sequence into a single, new symbol. The process is repeated until a specified number of merges is reached or no more frequent pairs remain”.



BPE: An example



Dictionary

5 low
2 lower
6 newest
3 widest

5 low
2 lower
6 new**est**
3 wide**st**

5 low
2 lower
6 new**est**
3 wide**est**

Vocabulary

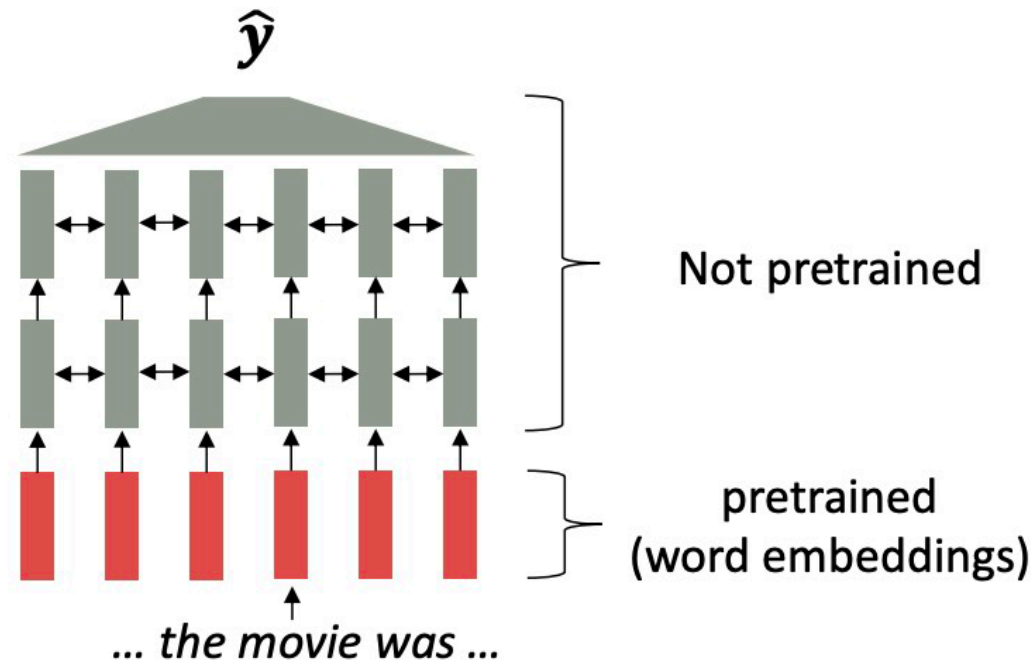
l, o, w, e, r, n, w, s, t, i, d

l, o, w, e, r, n, w, s, t, i, d, **es**

l, o, w, e, r, n, w, s, t, i, d, es, **est**

Common words end up being a part of the subword vocabulary, while rarer words are split into (sometimes intuitive, sometimes not) components.

	word		vocab mapping	embedding
Common words	hat	→	hat	
	learn	→	learn	
Variations	taaaaasty	→	taa## aaa## sty	  
misspellings	laern	→	la## ern##	 
novel items	Transformerify	→	Transformer## ify	 

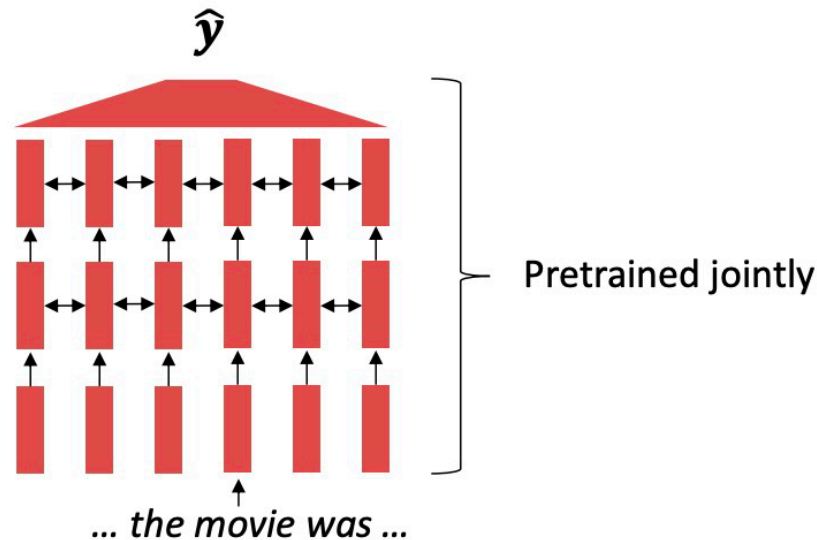


[Recall, *movie* gets the same word embedding,
no matter what sentence it shows up in]

Potential Issues:

1. The training data we have for our downstream task (like question answering) must be sufficient to teach all contextual aspects of language.
2. Most of the parameters in our network are randomly initialized!

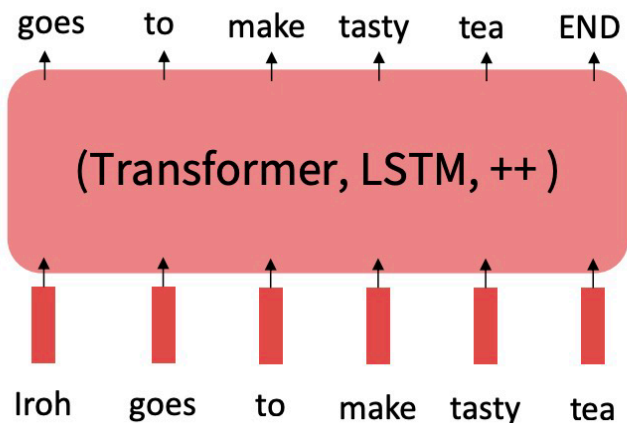
1. **(Almost) all parameters** in the networks are **initialized via pretraining**.
2. Pretraining methods hide parts of the input from the model, and train the model to reconstruct those parts.



[This model has learned how to represent entire sentences through pretraining]

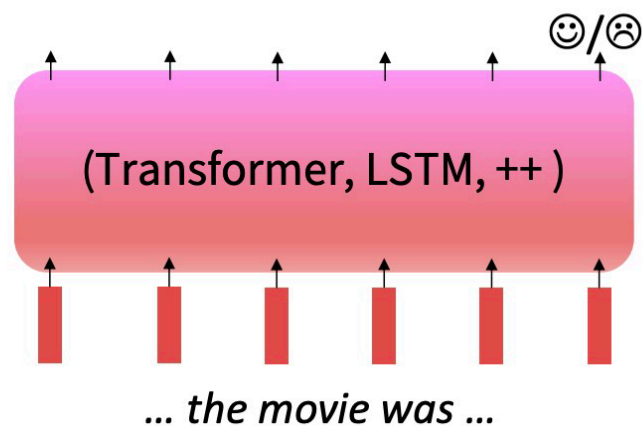
Step 1: Pretrain (on language modeling)

Lots of text; learn general things!



Step 2: Finetune (on your task)

Not many labels; adapt to the task!

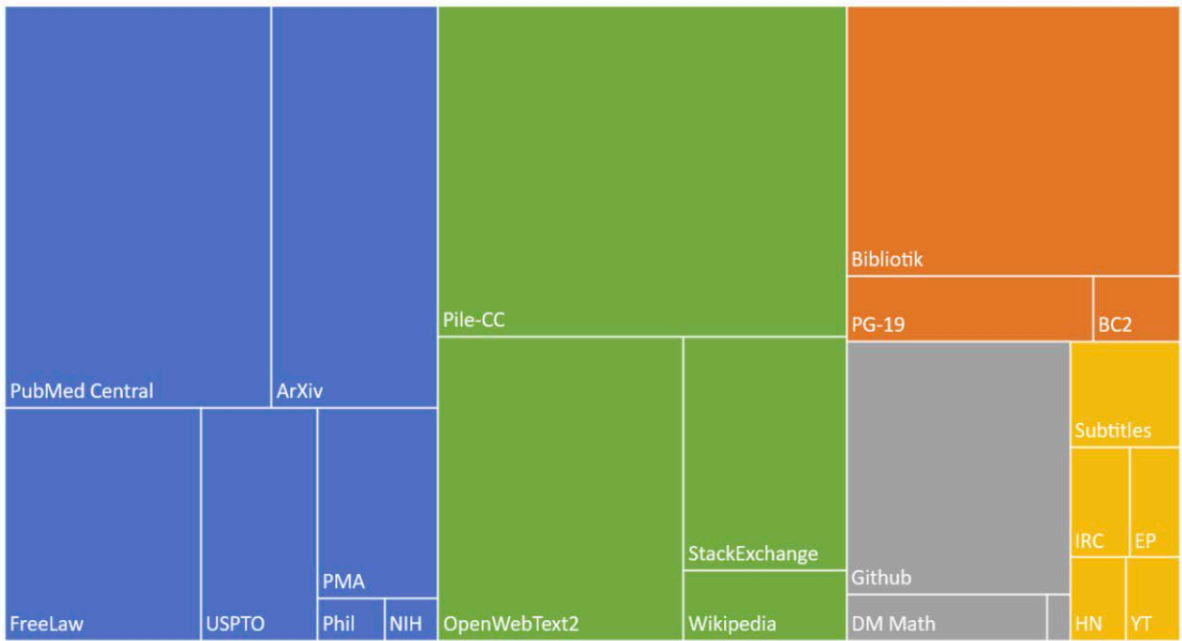




What Data for Pretraining?

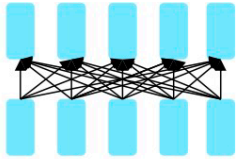
Composition of the Pile by Category

■ Academic ■ Internet ■ Prose ■ Dialogue ■ Misc



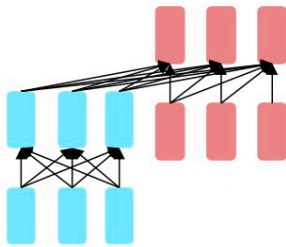
Model	Training Data
BERT	BookCorpus, English Wikipedia
GPT-1	BookCorpus
GPT-3	CommonCrawl, WebText, English Wikipedia, and 2 book databases ("Books 1" and "Books 2")
GPT-3.5+	Undisclosed

Three Types of Pretraining Architectures



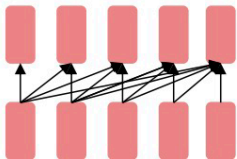
Encoders

- Gets bidirectional context – can condition on future!
- How do we train them to build strong representations?



**Encoder-
Decoders**

- Good parts of decoders and encoders?
- What's the best way to pretrain them?



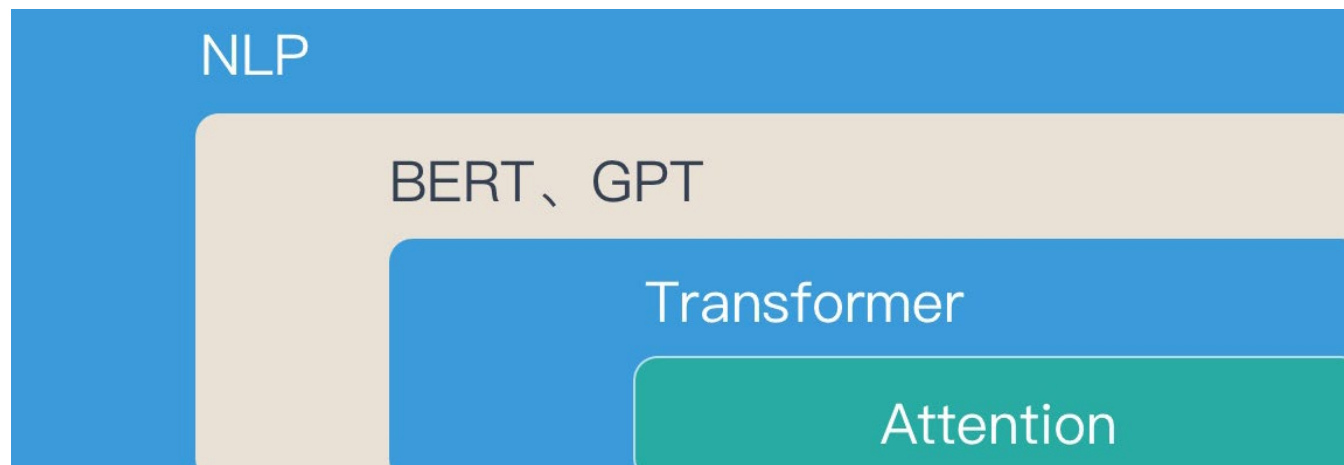
Decoders

- Language models! What we've seen so far.
- Nice to generate from; can't condition on future words

BERT

Devlin, J., Chang, M. W., Lee, K., & Toutanova, K. (2019). Bert: Pre-training of deep bidirectional transformers for language understanding. NAACL.

Google citation: **127375**



Suggested reading: [The Illustrated BERT, ELMo, and co.](#)

An overview of BERT



1 - Semi-supervised training on large amounts of text (books, wikipedia..etc).

The model is trained on a certain task that enables it to grasp patterns in language. By the end of the training process, BERT has language-processing abilities capable of empowering many models we later need to build and train in a supervised way.

Semi-supervised Learning Step

Model:



Dataset:



Objective:

Predict the masked word
(language modeling)

2 - Supervised training on a specific task with a labeled dataset.

Supervised Learning Step

Model:
(pre-trained in step #1)

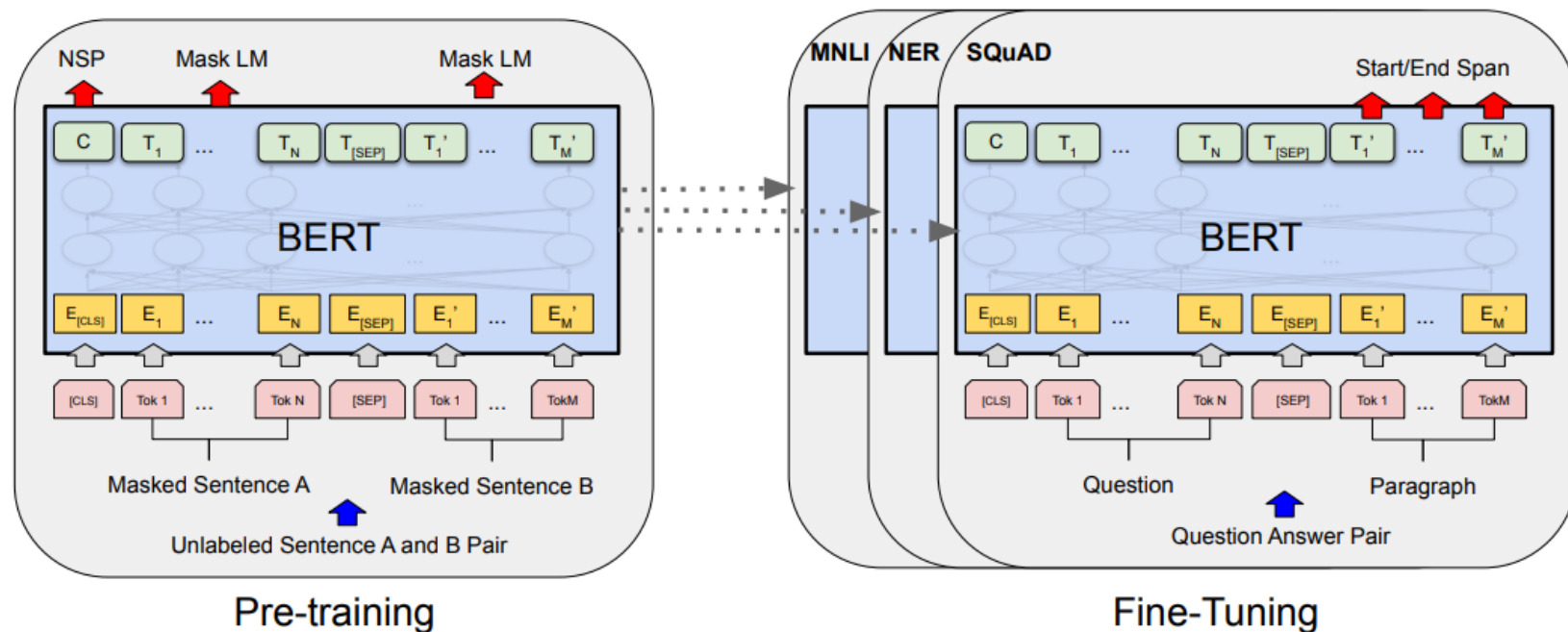


Classifier

75% Spam
25% Not Spam

Dataset:

Email message	Class
Buy these pills	Spam
Win cash prizes	Spam
Dear Mr. Atreides, please find attached...	Not Spam



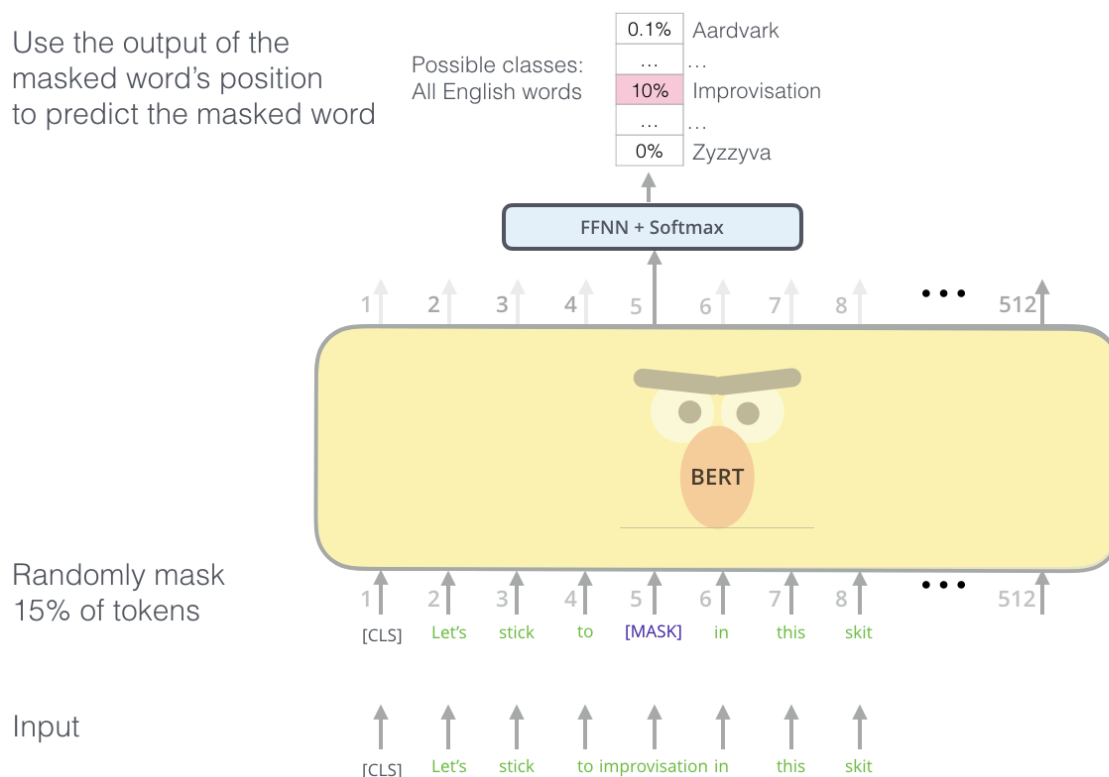
Pretraining with encoders only! But encoders get bidirectional context, so we can't do language modeling!

Ideas for Pretraining BERT: Masking



replace some fraction of words in the input with a special [MASK] token; predict these words. Masking 15% words randomly.

- Replace input word with [MASK] 80% of the time.
- Replace input word with a random token 10% of the time.
- Leave input word unchanged 10% of the time (but still predict it!).



Ideas for Pretraining BERT: Masking



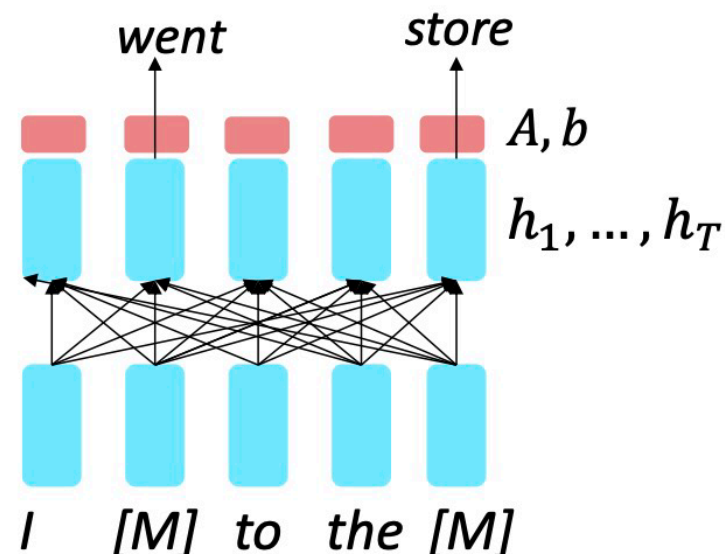
replace some fraction of words in the input with a special [MASK] token; predict these words.

$$h_1, \dots, h_T = \text{Encoder}(w_1, \dots, w_T)$$
$$y_i \sim Ah_i + b$$

If $\tilde{x}$ is the masked version of x , we will learn

$$p_\theta(x|\tilde{x})$$

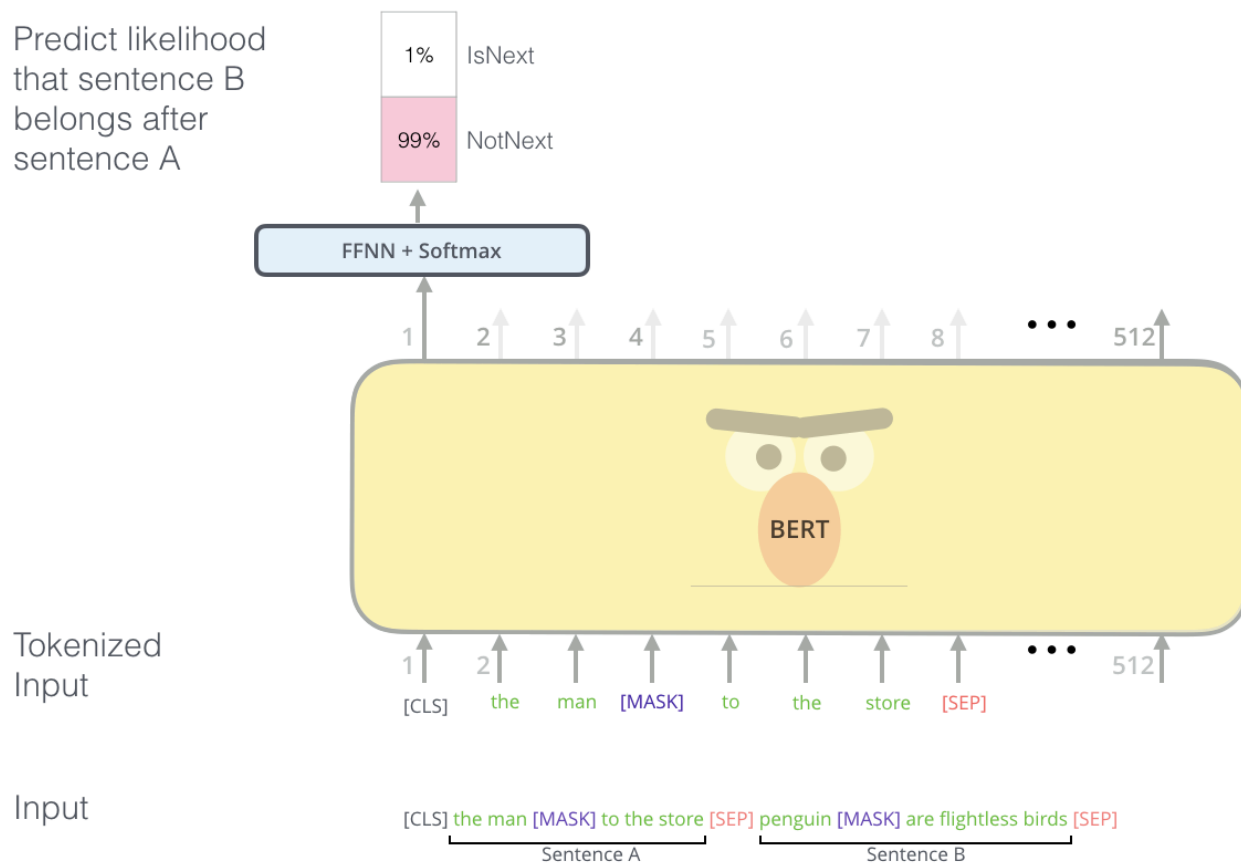
via the **masked Language Model**.



Ideas for Pretraining BERT: Two Sentences Task



To make BERT better at handling relationships between multiple sentences, the pre-training process includes an additional task: Given two sentences (A and B), is B likely to be the sentence that follows A, or not?

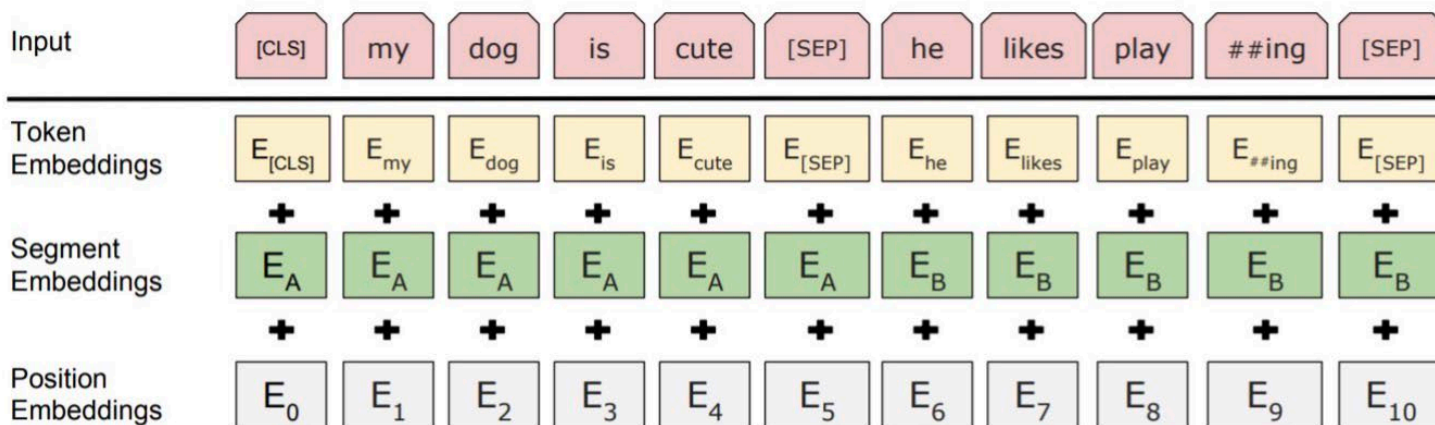


Ideas for Pretraining BERT: Two Sentences Task



To make BERT better at handling relationships between multiple sentences, the pre-training process includes an additional task: Given two sentences (A and B), is B likely to be the sentence that follows A, or not?

The pretraining input to BERT was two separate contiguous chunks of text:





- Two models were released:
 - BERT-base: 12 layers, 768-dim hidden states, 12 attention heads, 110 million params.
 - BERT-large: 24 layers, 1024-dim hidden states, 16 attention heads, 340 million params.
- Trained on:
 - BooksCorpus (800 million words)
 - English Wikipedia (2,500 million words)
- Pretraining is expensive and impractical on a single GPU.
 - BERT was pretrained with 64 TPU chips for a total of 4 days.
 - (TPUs are special tensor operation acceleration hardware)
- Finetuning is practical and common on a single GPU
 - “Pretrain once, finetune many times.”

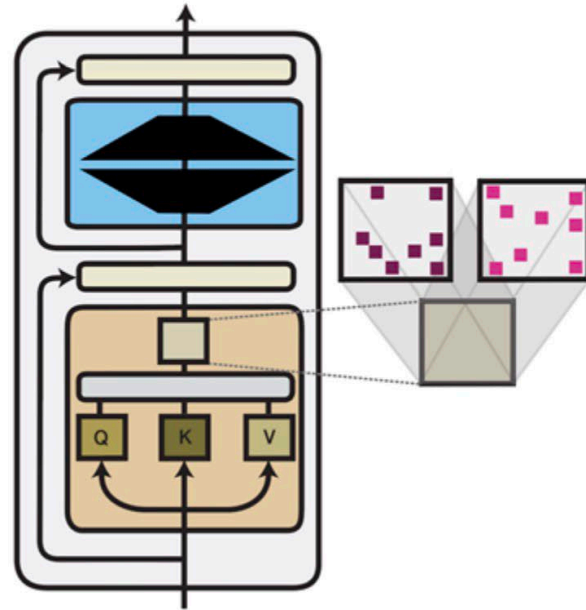
BERT was massively popular and hugely versatile; finetuning BERT led to new state-of-the-art results on a broad range of tasks.

- **QQP**: Quora Question Pairs (detect paraphrase questions)
- **QNLI**: natural language inference over question answering data
- **SST-2**: sentiment analysis
- **CoLA**: corpus of linguistic acceptability (detect whether sentences are grammatical.)
- **STS-B**: semantic textual similarity
- **MRPC**: microsoft paraphrase corpus
- **RTE**: a small natural language inference corpus

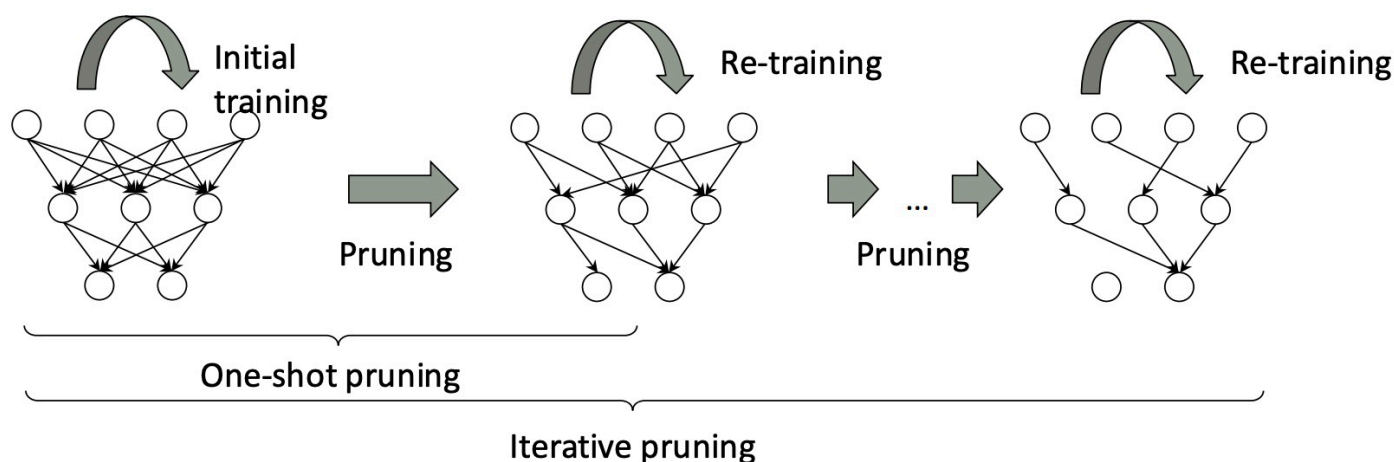
System	MNLI-(m/mm) 392k	QQP 363k	QNLI 108k	SST-2 67k	CoLA 8.5k	STS-B 5.7k	MRPC 3.5k	RTE 2.5k	Average -
Pre-OpenAI SOTA	80.6/80.1	66.1	82.3	93.2	35.0	81.0	86.0	61.7	74.0
BiLSTM+ELMo+Attn	76.4/76.1	64.8	79.8	90.4	36.0	73.3	84.9	56.8	71.0
OpenAI GPT	82.1/81.4	70.3	87.4	91.3	45.4	80.0	82.3	56.0	75.1
BERT _{BASE}	84.6/83.4	71.2	90.5	93.5	52.1	85.8	88.9	66.4	79.6
BERT _{LARGE}	86.7/85.9	72.1	92.7	94.9	60.5	86.5	89.3	70.1	82.1

Parameter-Efficient Fine-Tuning

- Sparse Subnetworks
- Low-rank Composition



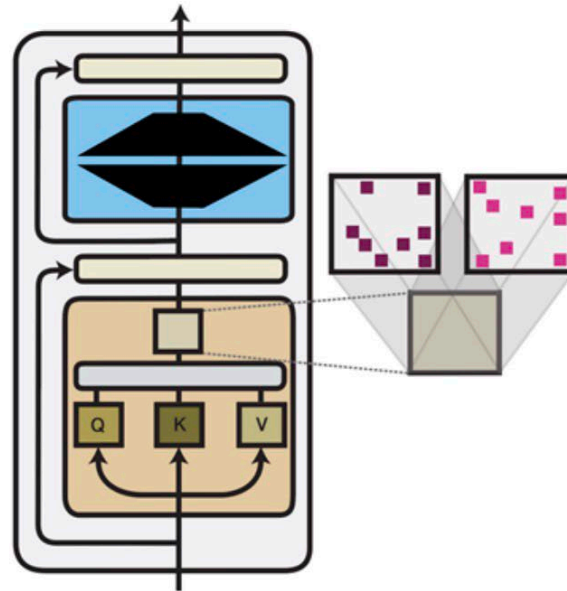
- During pruning, a fraction of the lowest-magnitude weights are removed
- The non-pruned weights are re-trained
- Pruning for multiple iterations is more common ([Frankle & Carbin, 2019](#))



The Lottery Ticket Hypothesis

- Dense, randomly-initialized models **contain subnetworks** (“winning tickets”) that—when trained in isolation—**reach test accuracy comparable to the original network** in a similar number of iterations ([Frankle & Carbin, 2019](#))

- ✓ Sparse Subnetworks
- Low-rank Composition



- Assume we have a pre-trained autoregressive language model $P_\phi(y|x)$
 - E.g., GPT based on Transformer
- Adapt this pretrained model to downstream tasks (e.g., summarization, NL2SQL, reading comprehension)
 - Training dataset of context-target pairs $\{(x_i, y_i)\}_{i=1, \dots, N}$
- During full fine-tuning, we update ϕ_o to $\phi_o + \Delta\phi$ by following the gradient to maximize the conditional language modeling objective

$$\max_{\phi} \sum_{(x,y)} \sum_{t=1}^{|y|} \log(P_\phi(y_t|x, y_{<t}))$$

Hu, E. J., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., & Chen, W. LoRA: Low-Rank Adaptation of Large Language Models. In International Conference on Learning Representations, 2021.

Google citation: **12416**

- For each downstream task, we learn a different set of parameters $\Delta\phi$
 - $|\Delta\phi| = |\phi_o|$
 - GPT-3 has a $|\phi_o|$ of 175 billion
 - Expensive and challenging for storing and deploying many independent instances
- **Key idea:** encode the task-specific parameter increment $\Delta\phi = \Delta\phi(\Theta)$ by a smaller-sized set of parameters Θ , $|\Theta| \ll |\phi_o|$
- The task of finding $\Delta\phi$ becomes optimizing over Θ

$$\max_{\Theta} \sum_{(x,y)} \sum_{t=1}^{|y|} \log(P_{\phi_o + \Delta\phi(\Theta)}(y_t|x, y_{<t}))$$

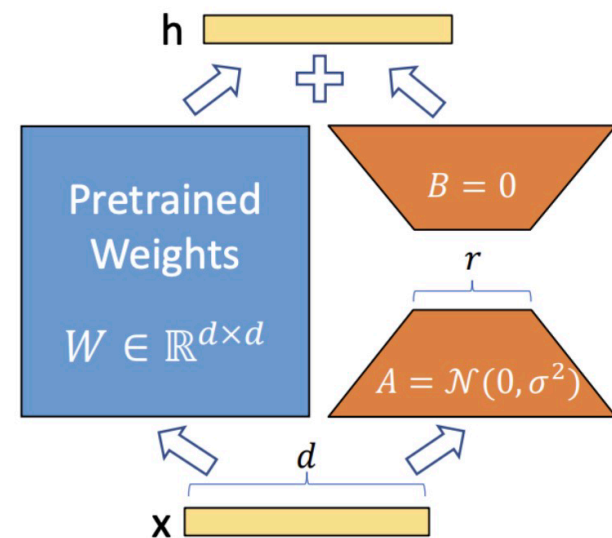
- Updates to the weights have a low “intrinsic rank” during adaptation (Aghajanyan et al. 2020)

- $W_0 \in \mathbb{R}^{d \times k}$: a pretrained weight matrix
- Constrain its update with a low-rank decomposition:

$$W_0 + \Delta W = W_0 + \alpha BA$$

where $B \in \mathbb{R}^{d \times r}$, $A \in \mathbb{R}^{r \times k}$, $r \ll \min(d, k)$

- α is the tradeoff between pre-trained “knowledge” and task-specific “knowledge”
- Only A and B contain **trainable** parameters



- **Few-shot learner: In-Context Learning**
- Instruction Fine-tuning
- RLHF

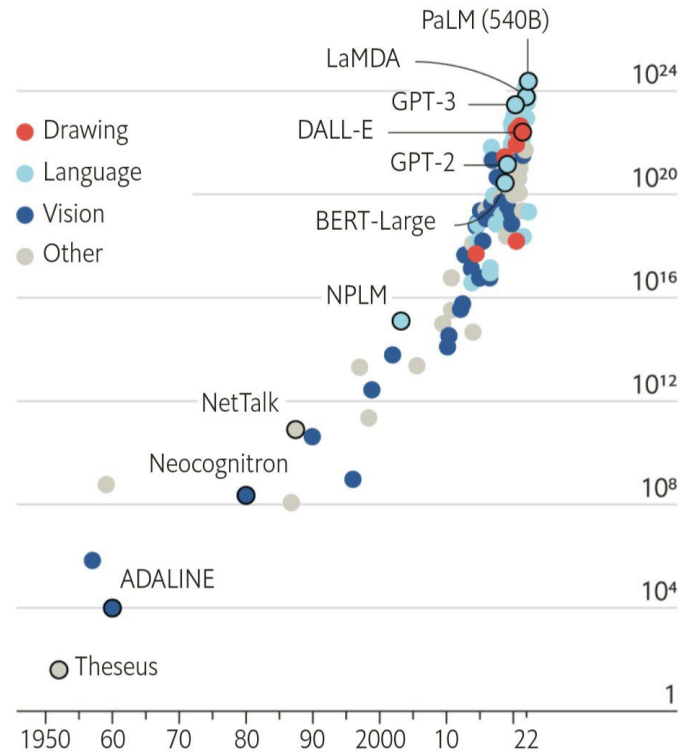
The Blessing of Scale in AI



The blessings of scale

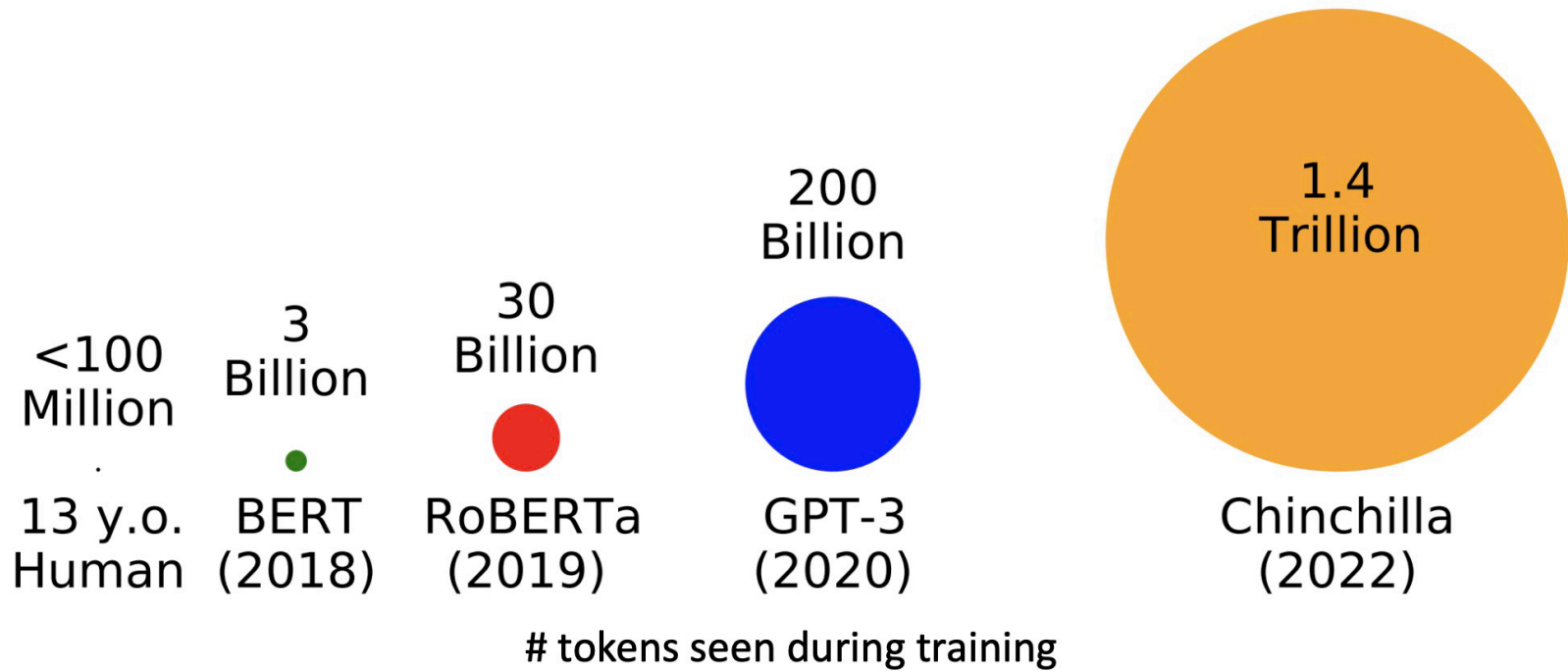
AI training runs, estimated computing resources used

Floating-point operations, selected systems, by type, log scale



Sources: "Compute trends across three eras of machine learning", by J. Sevilla et al., arXiv, 2022; Our World in Data

Tokens Seen by Models in Training



Math Problems

We can describe circles in the xy -plane using equations in terms of x and y . Circle equations questions require us to understand the connection between these equations and the features of circles.

For example, the equation $(x + 2)^2 + (y - 3)^2 = 4^2$ is graphed in the xy -plane below. It is a circle with a center at $(-2, 3)$ and a radius of 4.

Coding Problems

```
5 // Determine whether the sentiment of text is positive
6 // Use a web service
7 async function isPositive(text: string): Promise<boolean> {
8   const response = await fetch(`http://text-processing.com/api/sentiment/`, {
9     method: "POST",
10    body: `text=${text}`,
11    headers: {
12      "Content-Type": "application/x-www-form-urlencoded",
13    },
14  });
15  const json = await response.json();
16  return json.label === "pos";
17 }
```

Planning

I need to throw a dinner party for 6 people who are vegetarian. Can you suggest a 3-course menu with a chocolate dessert?

Sure, I can suggest a 3-course menu with a chocolate dessert for your vegetarian dinner party. Here are some possible options:

- Starter: **Wild Mushroom Tartlets with Onion Sauce**¹ or **Vegan Popcorn Tofu Nuggets**²
- Main: **Vegan Butternut Squash Mac**² or **Vegetarian Three-Bean Chili**³
- Dessert: **Chocolate Lava Cake**⁴ or **Chocolate Pasta with Chocolate Hazelnut Cream Sauce, White Chocolate Shavings, and Fresh Berries**⁴

I hope this helps you plan your party. Enjoy! 😊

Learn more: [1. booths.co.uk](https://1.booths.co.uk) [+10 more](#)

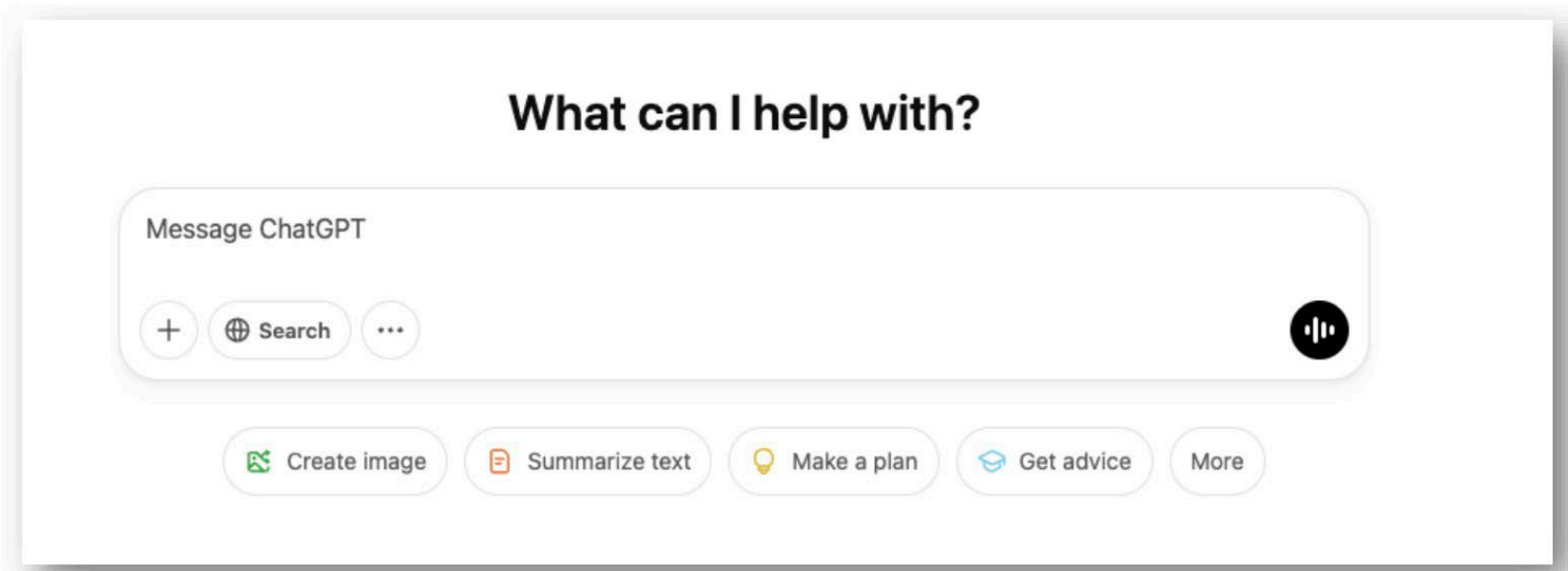
Language Models as Multi-task Assistants?



How do we get from

The Hong Kong Polytechnic University is located in _____

to



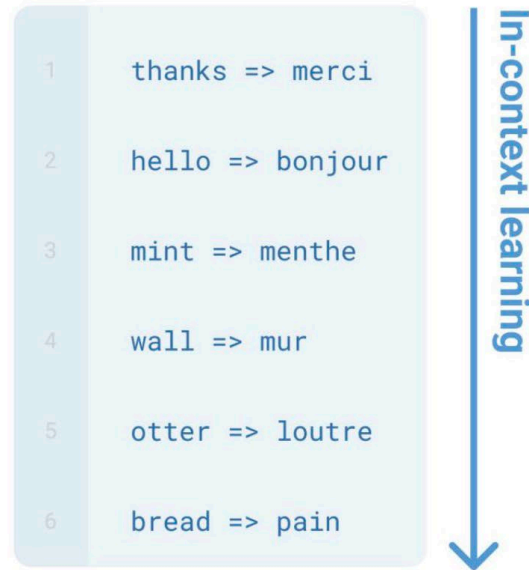
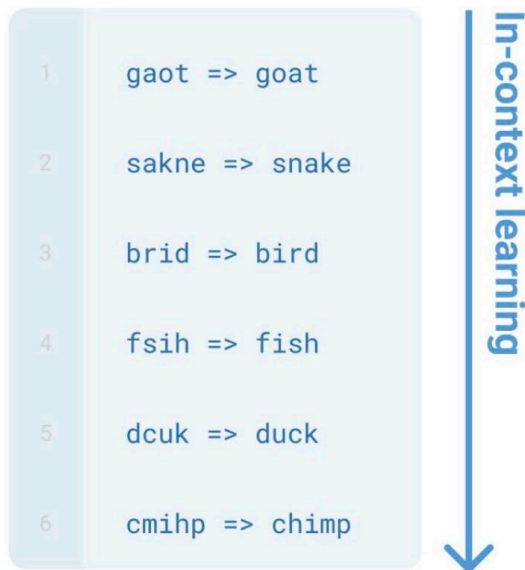
Brown, T., Mann, B., Ryder, N., Subbiah, M., Kaplan, J. D., Dhariwal, P., ... & Amodei, D. (2020). Language models are few-shot learners. *Advances in neural information processing systems*, 33, 1877-1901.

Google citation: **43367**

GPT-3 (175B parameters; [Brown et al., 2020](#))

- Another increase in size (1.5B -> **175B**)
- and data (40GB -> **over 600GB**)

- Specify a task by simply **prepending examples of the task before your example**
- Also called **in-context learning**, to stress that *no gradient updates* are performed when learning a new task (there is a separate literature on few-shot learning with gradient updates)



Emergent Few-shot Learning

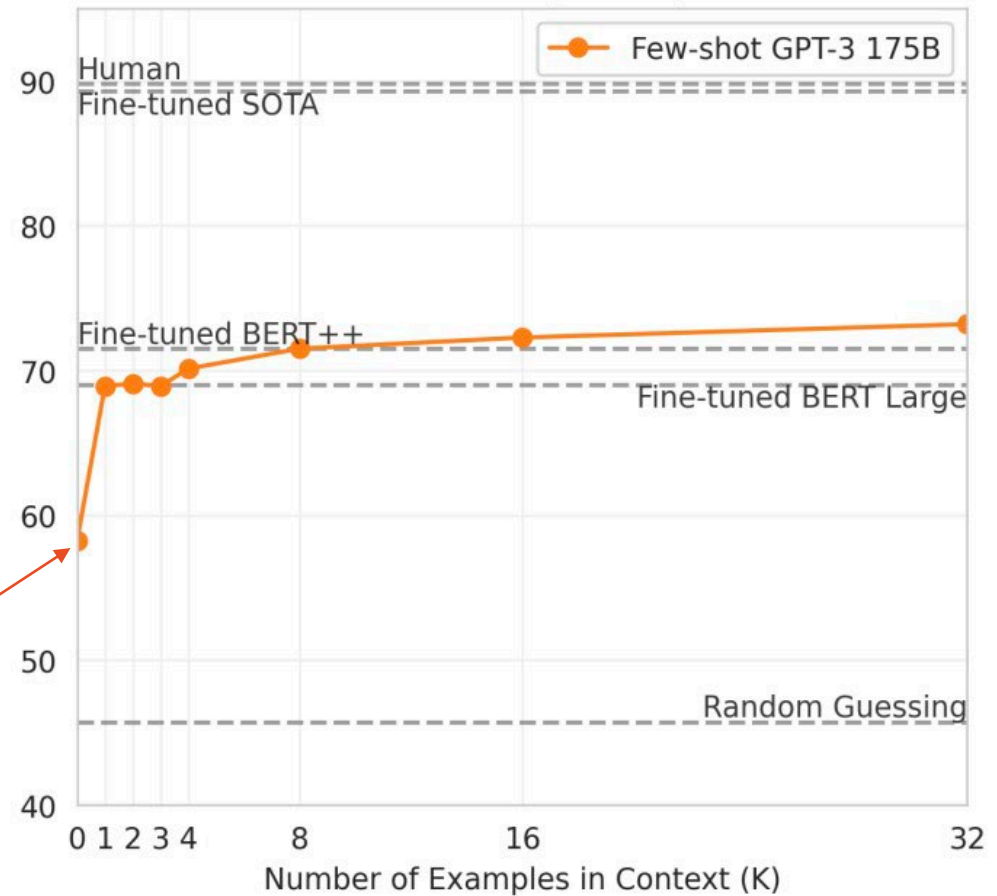


Zero-shot

1 Translate English to French:

2 cheese =>

In-Context Learning on SuperGLUE



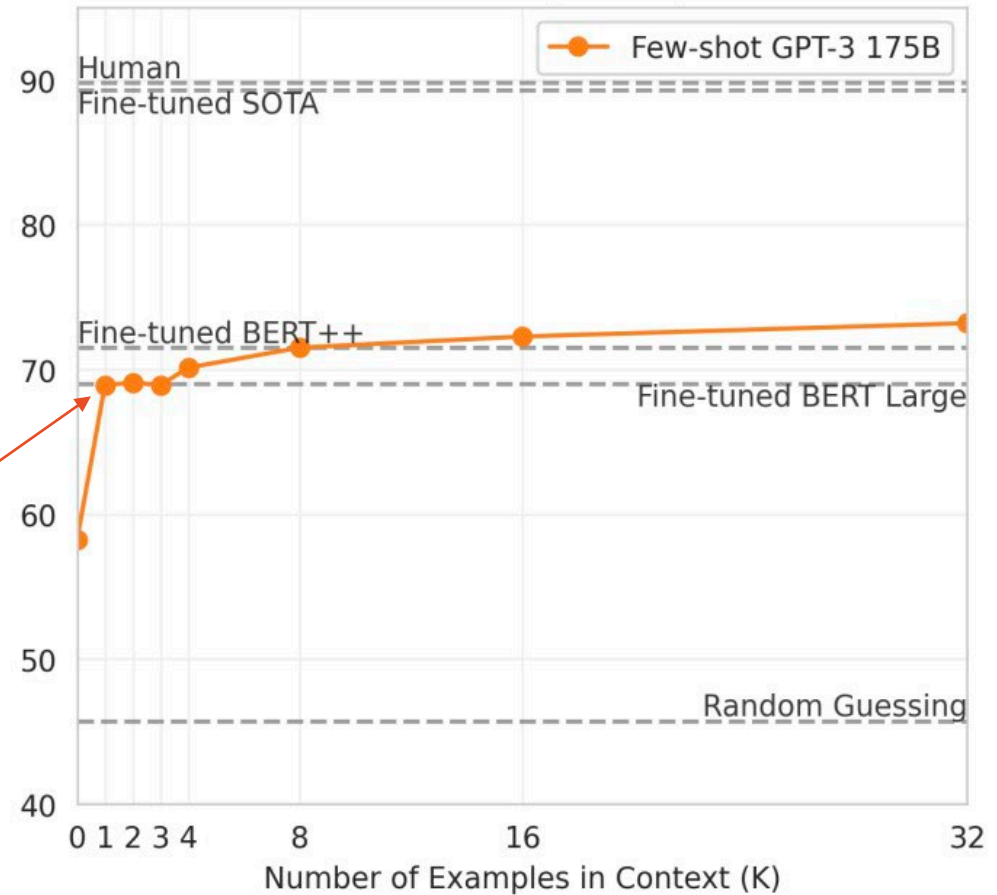
Emergent Few-shot Learning



One-shot

```
1 Translate English to French:
2 sea otter => loutre de mer
3 cheese =>
```

In-Context Learning on SuperGLUE



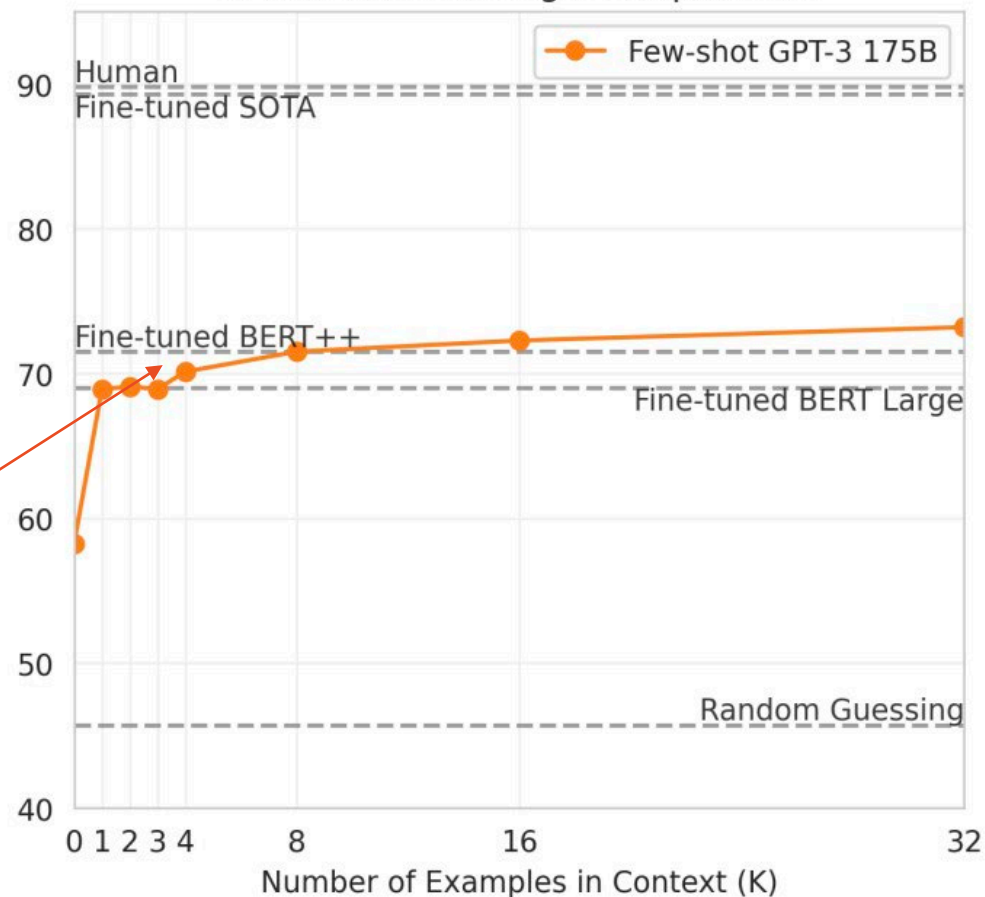
Emergent Few-shot Learning



Few-shot

1 Translate English to French: ←
2 sea otter => loutre de mer ←
3 peppermint => menthe poivrée ←
4 plush girafe => girafe peluche ←
5 cheese => ←

In-Context Learning on SuperGLUE



Zero/few-shot prompting

1 Translate English to French: ←
2 sea otter => loutre de mer ←
3 peppermint => menthe poivrée ←
4 plush girafe => girafe peluche ←
5 cheese => ←

Traditional fine-tuning

1 sea otter => loutre de mer ←



gradient update



1 peppermint => menthe poivrée ←



gradient update



1 cheese => ←

Some tasks seem too hard for even large LMs to learn through prompting alone.
Especially tasks involving **richer, multi-step reasoning**.
(Humans struggle at these tasks too!)

$$\begin{array}{rcl} 19583 & + & 29534 = 49117 \\ 98394 & + & 49384 = 147778 \\ 29382 & + & 12347 = 41729 \\ 93847 & + & 39299 = ? \end{array}$$

Solution: Change the prompt!!!

Wei, J., Wang, X., Schuurmans, D., Bosma, M., Xia, F., Chi, E., ... & Zhou, D. (2022). Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems*, 35, 24824-24837.

Google citation: **12853**

Standard Prompting

Model Input

Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?

A: The answer is 11.

Q: The cafeteria had 23 apples. If they used 20 to make lunch and bought 6 more, how many apples do they have?

Model Output

A: The answer is 27. ❌

Chain-of-Thought Prompting

Model Input

Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?

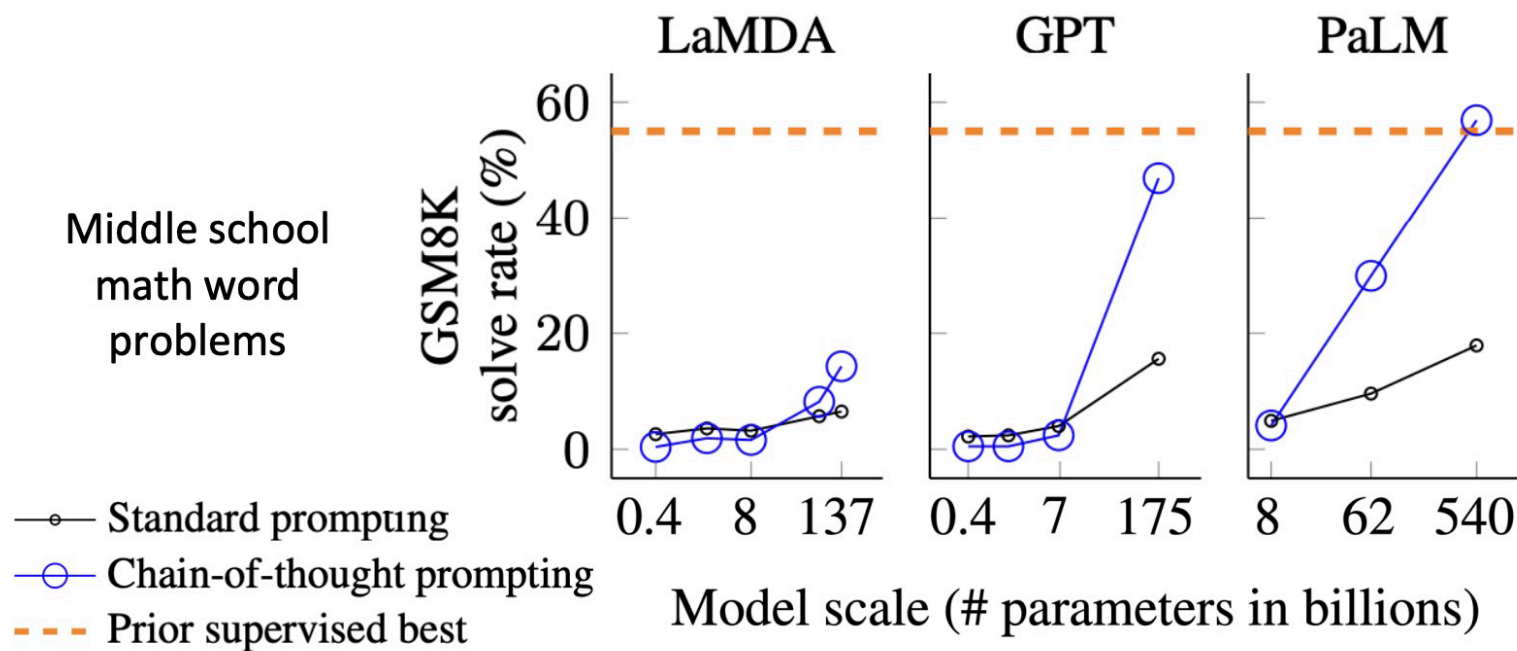
A: Roger started with 5 balls. 2 cans of 3 tennis balls each is 6 tennis balls. $5 + 6 = 11$. The answer is 11.

Q: The cafeteria had 23 apples. If they used 20 to make lunch and bought 6 more, how many apples do they have?

Model Output

A: The cafeteria had 23 apples originally. They used 20 to make lunch. So they had $23 - 20 = 3$. They bought 6 more apples, so they have $3 + 6 = 9$. The answer is 9. ✅

Chain-of-Thought Prompting



Model Input

Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?

A: Roger started with 5 balls. 2 cans of 3 tennis balls each is 6 tennis balls. $5 + 6 = 11$. The answer is 11.

Q: The cafeteria had 23 apples. If they used 20 to make lunch and bought 6 more, how many apples do they have?

Model Output

A: The cafeteria had 23 apples originally. They used 20 to make lunch. So they had $23 - 20 = 3$. They bought 6 more apples, so they have $3 + 6 = 9$. The answer is 9. ✓

Q: A juggler can juggle 16 balls. Half of the balls are golf balls, and half of the golf balls are blue. How many blue golf balls are there?

A: **Let's think step by step.** There are 16 balls in total. Half of the balls are golf balls. That means there are 8 golf balls. Half of the golf balls are blue. That means there are 4 blue golf balls. ✓

No.	Category	Zero-shot CoT Trigger Prompt	Accuracy
1	LM-Designed	Let's work this out in a step by step way to be sure we have the right answer.	82.0
2	Human-Designed 	Let's think step by step. (*1)	78.7
3		First, (*2)	77.3
4		Let's think about this logically.	74.5
5		Let's solve this problem by splitting it into steps. (*3)	72.2
6		Let's be realistic and think step by step.	70.8
7		Let's think like a detective step by step.	70.3
8		Let's think	57.5
9		Before we dive into the answer,	55.7
10		The answer is after the proof.	45.7
-		(Zero-shot)	17.7



Prompt engineering

文A 5 languages ▾

Article [Talk](#)

More ▾

From Wikipedia, the free encyclopedia

Prompt engineering is a concept in [artificial intelligence](#), particularly [natural language processing](#) (NLP). In prompt engineering, the description of the task is

Prompt Engineer and Librarian

APPLY FOR THIS JOB

SAN FRANCISCO, CA / PRODUCT / FULL-TIME / HYBRID

Pros:

- **No finetuning needed, prompt engineering (e.g. CoT) can improve performance.**

Cons:

- **Limits to what you can fit in context**
- **Complex tasks will probably need gradient steps**



- Few-shot learner: In-Context Learning
- **Instruction Fine-tuning**
- RLHF

Part of the contents are inspired and adopted from Stanford CS224n.

Ouyang, L., Wu, J., Jiang, X., Almeida, D., Wainwright, C., Mishkin, P., ... & Lowe, R. (2022). Training language models to follow instructions with human feedback. *Advances in neural information processing systems*, 35, 27730-27744.
Google citation: **13430**

PROMPT *Explain the moon landing to a 6 year old in a few sentences.*

COMPLETION GPT-3

Explain the theory of gravity to a 6 year old.

Explain the theory of relativity to a 6 year old in a few sentences.

Explain the big bang theory to a 6 year old.

Explain evolution to a 6 year old.

Language models are not *aligned* with user intent [[Ouyang et al., 2022](#)]

Finetuning to the rescue!

PROMPT *Explain the moon landing to a 6 year old in a few sentences.*

COMPLETION **Human**

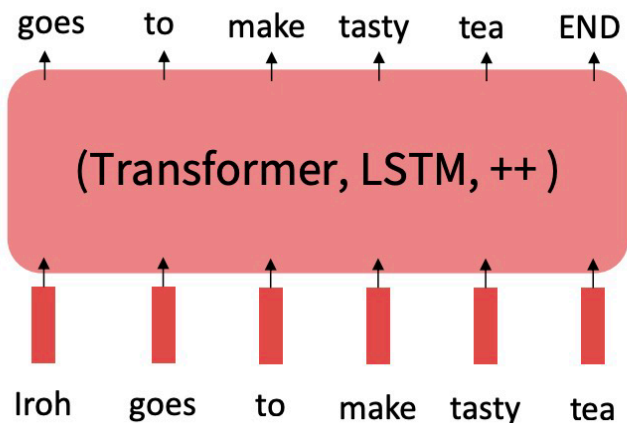
A giant rocket ship blasted off from Earth carrying astronauts to the moon. The astronauts landed their spaceship on the moon and walked around exploring the lunar surface. Then they returned safely back to Earth, bringing home moon rocks to show everyone.

Language models are not *aligned* with user intent [[Ouyang et al., 2022](#)]

Finetuning to the rescue!

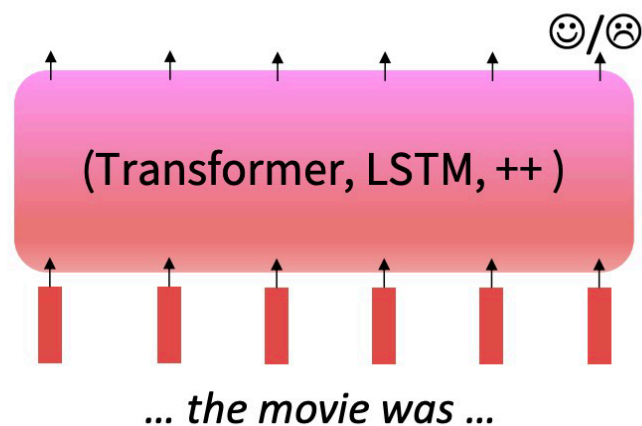
Step 1: Pretrain (on language modeling)

Lots of text; learn general things!



Step 2: Finetune (on your task)

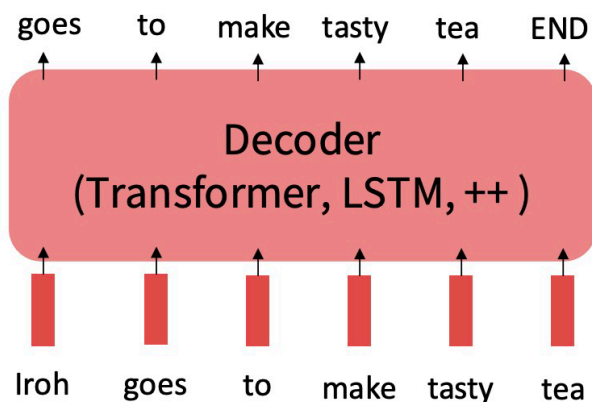
Not many labels; adapt to the task!



Pretraining can improve NLP applications by serving as parameter initialization.

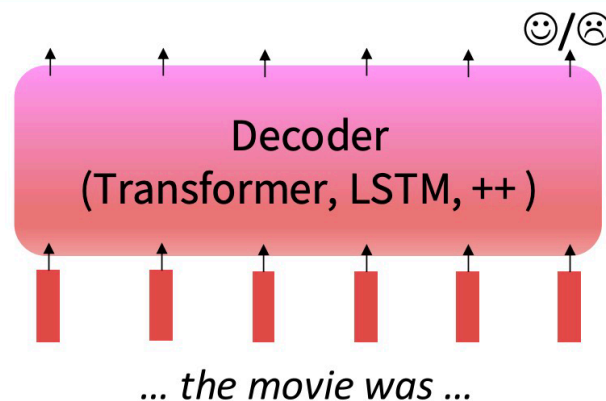
Step 1: Pretrain (on language modeling)

Lots of text; learn general things!

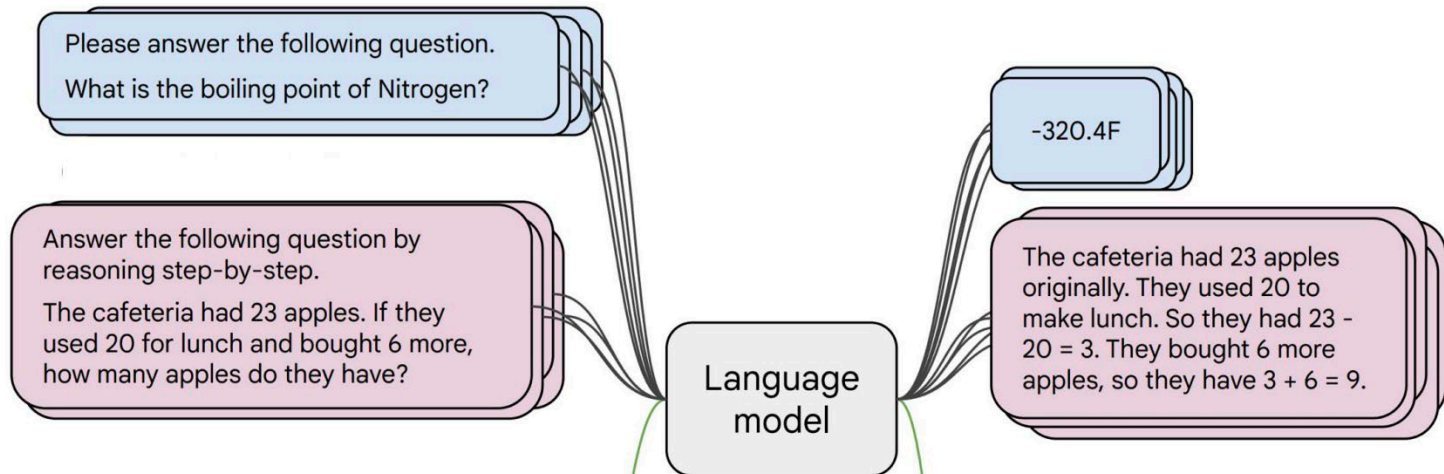


Step 2: Finetune (on **many tasks**)

~~Not~~ many labels; adapt to the tasks!



- **Collect examples** of (instruction, output) pairs across many tasks and finetune an LM



- **Evaluate on unseen tasks**



[FLAN-T5; [Chung et al., 2022](#)]

Instruction Pretraining?

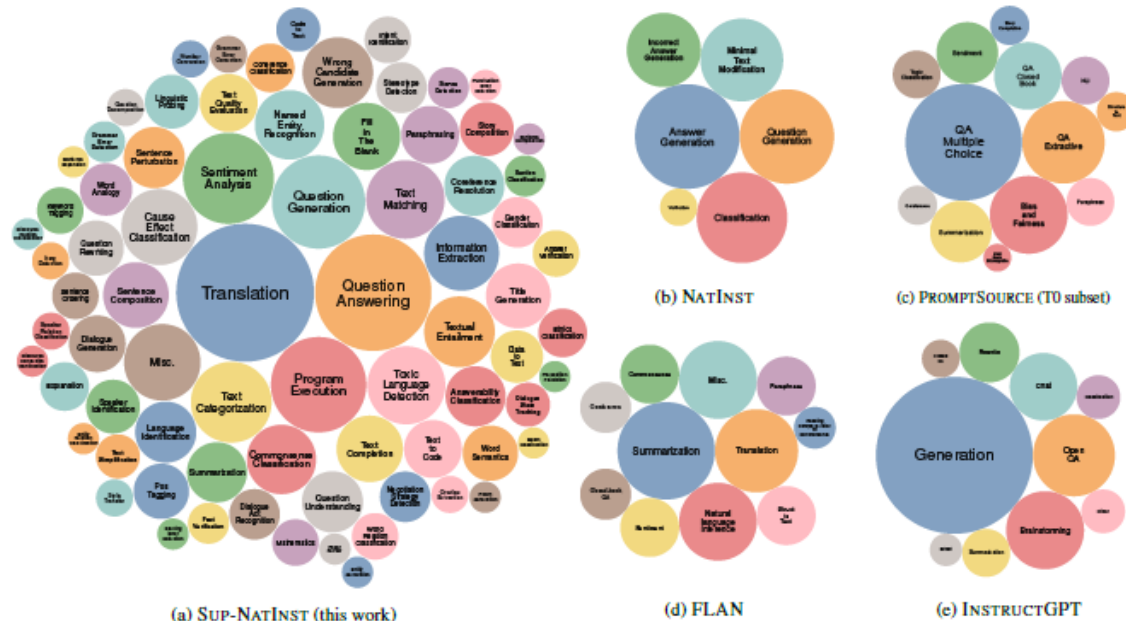


Figure 2: Compared to other datasets, **SUP-NATINST** covers a more diverse range of task types. InstructGPT reports a very coarse categorization of their task types. Bubble size represents the number of tasks of each type in log scale.

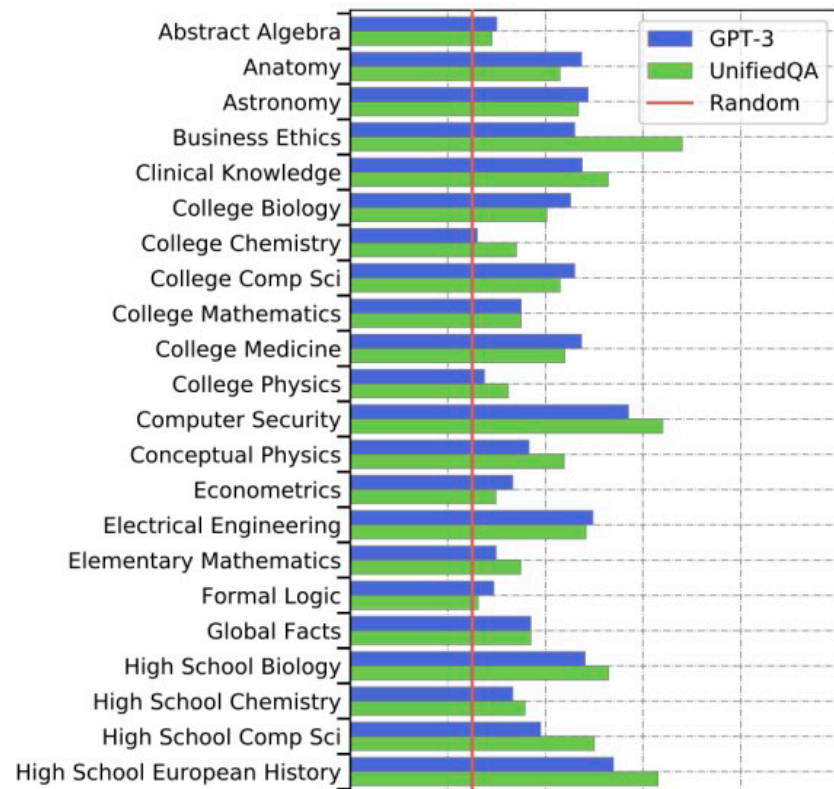
- As is usually the case, **data + model scale** is key for this to work!
- **Super-NaturalInstructions** dataset contains **over 1.6K tasks**, **3M+** examples
 - Classification, sequence tagging, rewriting, translation, QA...

Q: how do we evaluate such a model?

Massive Multitask Language Understanding (MMLU)

[[Hendrycks et al., 2021](#)]

New benchmarks for measuring LM performance on 57 diverse *knowledge intensive* tasks





Astronomy

What is true for a type-Ia supernova?

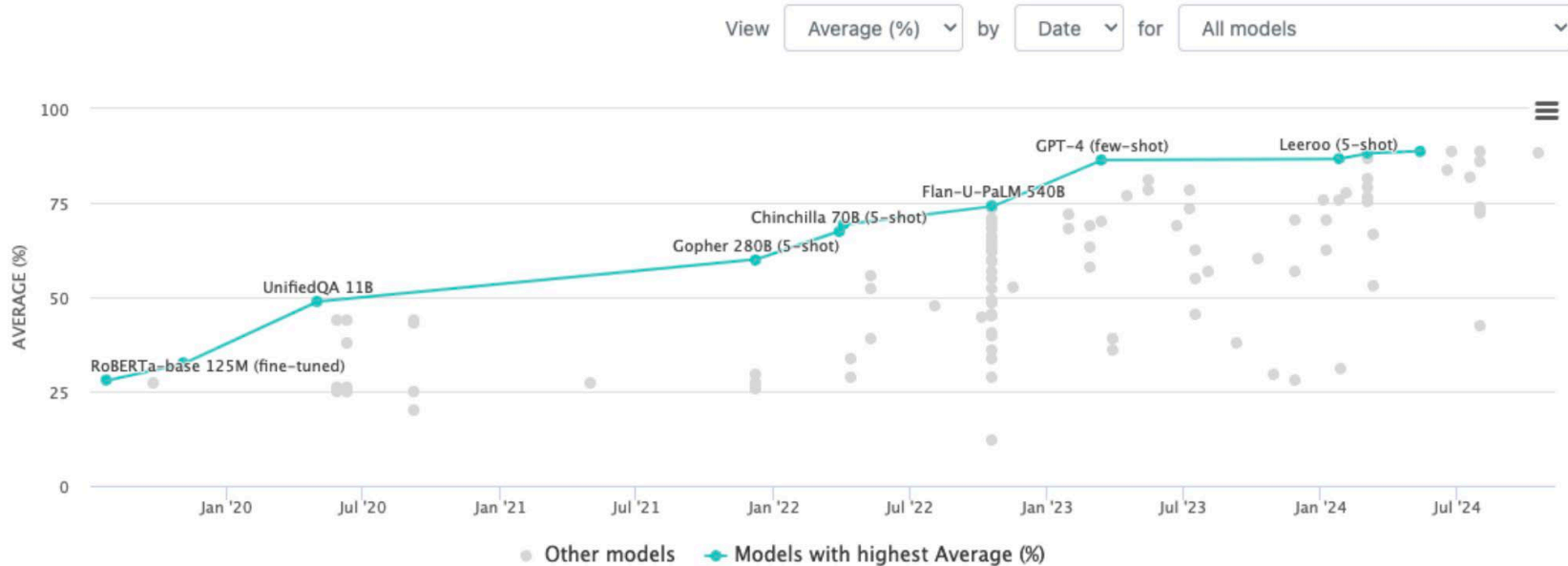
- A. This type occurs in binary systems.
- B. This type occurs in young galaxies.
- C. This type produces gamma-ray bursts.
- D. This type produces high amounts of X-rays.

High School Biology

In a population of giraffes, an environmental change occurs that favors individuals that are tallest. As a result, more of the taller individuals are able to obtain nutrients and survive to pass along their genetic information. This is an example of

- A. directional selection.
- B. stabilizing selection.
- C. sexual selection.
- D. disruptive selection

Progress on MMLU



- Rapid, impressive progress on challenging knowledge-intensive benchmarks

Model input (Disambiguation QA)

Q: In the following sentences, explain the antecedent of the pronoun (which thing the pronoun refers to), or state that it is ambiguous.

Sentence: The reporter and the chef will discuss their favorite dishes.

Options:

- (A) They will discuss the reporter's favorite dishes
- (B) They will discuss the chef's favorite dishes
- (C) Ambiguous

A: Let's think step by step.

Before instruction finetuning

The reporter and the chef will discuss their favorite dishes.

The reporter and the chef will discuss the reporter's favorite dishes.

The reporter and the chef will discuss the chef's favorite dishes.

The reporter and the chef will discuss the reporter's and the chef's favorite dishes.

✗ (doesn't answer question)

Model input (Disambiguation QA)

Q: In the following sentences, explain the antecedent of the pronoun (which thing the pronoun refers to), or state that it is ambiguous.

Sentence: The reporter and the chef will discuss their favorite dishes.

Options:

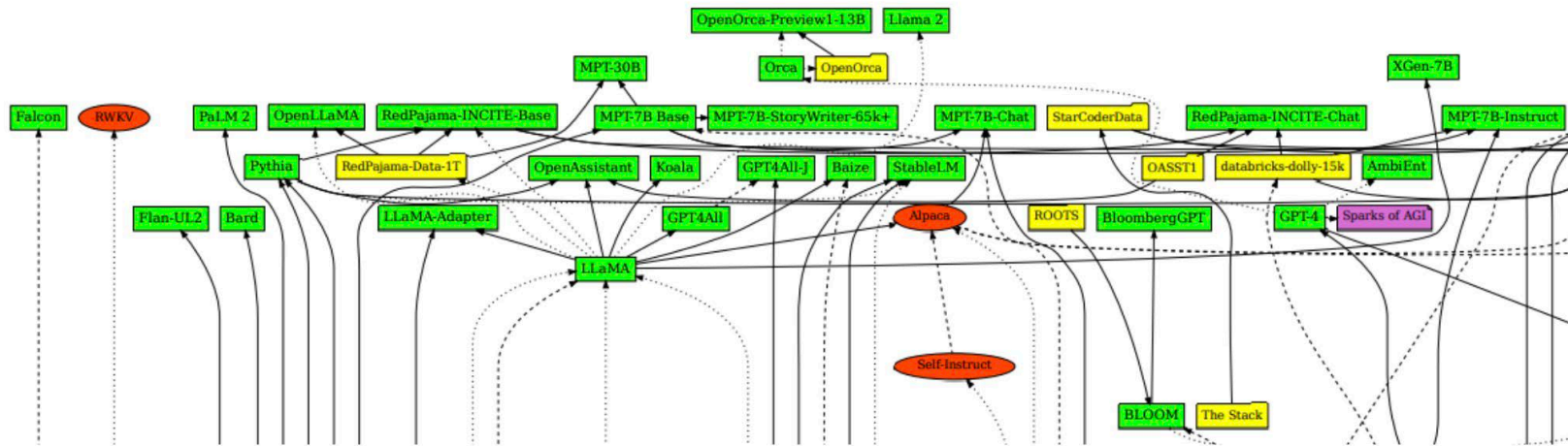
- (A) They will discuss the reporter's favorite dishes
- (B) They will discuss the chef's favorite dishes
- (C) Ambiguous

A: Let's think step by step.

After instruction finetuning

The reporter and the chef will discuss their favorite dishes does not indicate whose favorite dishes they will discuss. So, the answer is (C). ✓

A huge diversity of data



- The release of LLaMA led to open-source attempts to 'create' instruction tuning data

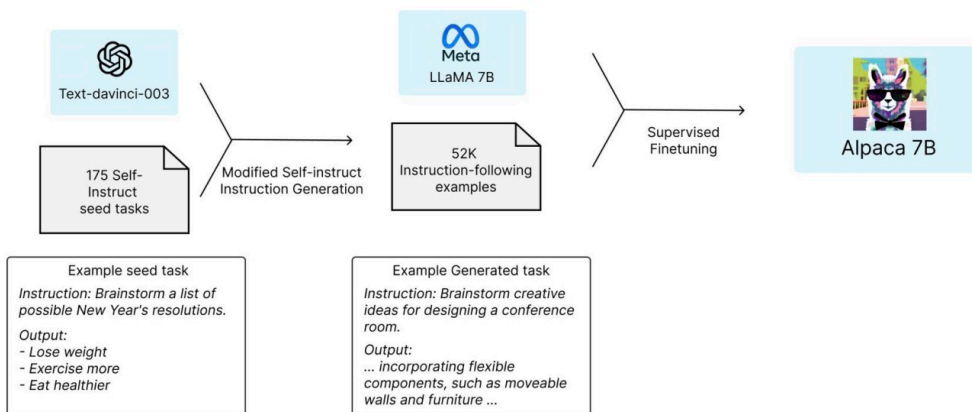
You Do Not Need Many Samples



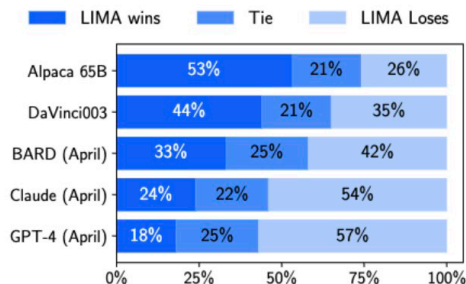
- Generate instructions, input, and output from a LM [\[Wang et al., 2022\]](#)

- Alpaca**: fine-tuned from the LLaMA 7B model on 52K instruction-following examples

- You don't need many samples to instruction tune (e.g., "*LIMA: Less Is More for Alignment*" [Zhou et al., 2023](#))



Source	#Examples
Training	
Stack Exchange (STEM)	200
Stack Exchange (Other)	200
wikiHow	200
Pushshift r/WritingPrompts	150
Natural Instructions	50
Paper Authors (Group A)	200

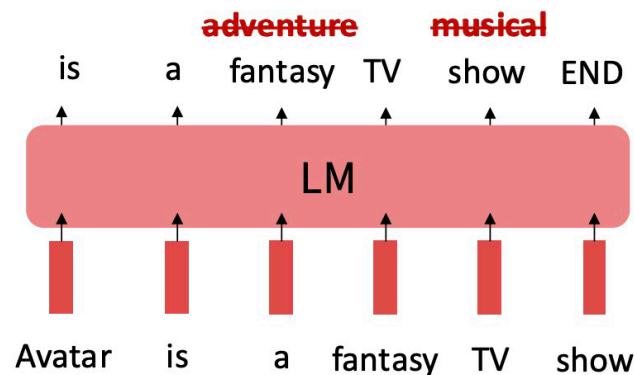




- Few-shot learner: In-Context Learning
- Instruction Fine-tuning
- **RLHF**

Part of the contents are inspired and adopted from Stanford CS224n.

- One limitation of instruction finetuning is obvious: it's **expensive** to collect ground-truth data for tasks.
- But there are other, subtler limitations too. Can you think of any?
- **Problem 1:** tasks like open-ended creative generation have no right answer.
 - *Write me a story about a dog and her pet grasshopper.*
- **Problem 2:** language modeling penalizes all token-level mistakes equally, but some errors are worse than others.
- Even with instruction finetuning, there is a mismatch between the LM objective and the objective of “satisfy human preferences”!
- Can we **explicitly attempt to satisfy human preferences**?



- Let's say we were training a language model on some task (e.g. summarization).
- For each LM sample s , imagine we had a way to obtain a *human reward* of that summary: $R(s) \in \mathbb{R}$, higher is better.

SAN FRANCISCO,
California (CNN) --
A magnitude 4.2
earthquake shook the
San Francisco

...
overturn unstable
objects.

An earthquake hit
San Francisco.
There was minor
property damage,
but no injuries.

$$s_1 \\ R(s_1) = 8.0$$

The Bay Area has
good weather but is
prone to
earthquakes and
wildfires.

$$s_2 \\ R(s_2) = 1.2$$

- Now we want to maximize the expected reward of samples from our LM:

$$\mathbb{E}_{\hat{s} \sim p_{\theta}(s)}[R(\hat{s})]$$

Note: for mathematical simplicity we're assuming only one "prompt"

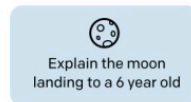
Reinforce Learning with Human Feedback Pipeline



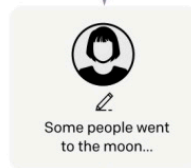
Step 1

Collect demonstration data, and train a supervised policy.

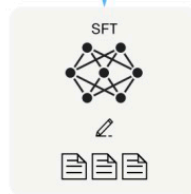
A prompt is sampled from our prompt dataset.



A labeler demonstrates the desired output behavior.



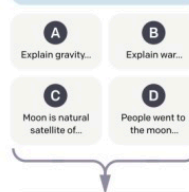
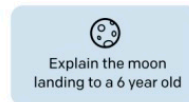
This data is used to fine-tune GPT-3 with supervised learning.



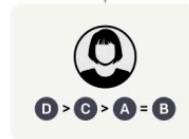
Step 2

Collect comparison data, and train a reward model.

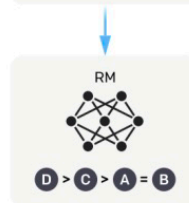
A prompt and several model outputs are sampled.



A labeler ranks the outputs from best to worst.



This data is used to train our reward model.



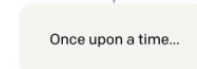
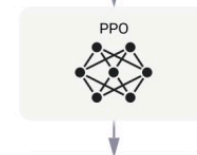
Step 3

Optimize a policy against the reward model using reinforcement learning.

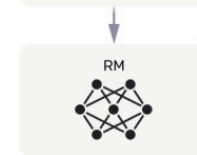
A new prompt is sampled from the dataset.



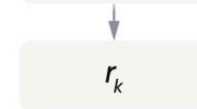
The policy generates an output.



The reward model calculates a reward for the output.



The reward is used to update the policy using PPO.



- First step: instruction tuning!
- Second + third steps: maximize reward (but how??)

- The field of **reinforcement learning (RL)** has studied these (and related) problems for many years now [[Williams, 1992](#); [Sutton and Barto, 1998](#)]
- Circa 2013: resurgence of interest in RL applied to deep learning, game-playing [[Mnih et al., 2013](#)]
- But the interest in applying RL to modern LMs is an even newer phenomenon [[Ziegler et al., 2019](#); [Stiennon et al., 2020](#); [Ouyang et al., 2022](#)]. **Why?**
 - RL w/ LMs has commonly been viewed as very hard to get right (still is!)
 - Newer advances in RL algorithms that work for large neural models, including language models (e.g. PPO; [[Schulman et al., 2017](#)])





<https://www.youtube.com/watch?v=3vFISl7qMFI>

RL-AlphaGo: <https://www.youtube.com/watch?v=4PyWLgrt7YY>

RL-ChatGPT and RLHF: <https://www.youtube.com/watch?v=WMmGzx-jWvs&t=101s>

- How do we actually change our LM parameters θ to maximize this?

$$\mathbb{E}_{\hat{s} \sim p_{\theta}(s)}[R(\hat{s})]$$

- Let's try doing gradient ascent!

$$\theta_{t+1} := \theta_t + \alpha \nabla_{\theta_t} \mathbb{E}_{\hat{s} \sim p_{\theta_t}(s)}[R(\hat{s})]$$

How do we estimate
this expectation??

What if our reward
function is non-
differentiable??

- **Policy gradient** methods in RL (e.g., REINFORCE; [[Williams, 1992](#)]) give us tools for estimating and optimizing this objective.

- We want to obtain

$$\nabla_{\theta} \mathbb{E}_{\hat{s} \sim p_{\theta}(s)} [R(\hat{s})] = \nabla_{\theta} \sum_s R(s) p_{\theta}(s) = \sum_s R(s) \nabla_{\theta} p_{\theta}(s)$$

(defn. of expectation) (linearity of gradient)

- Here we'll use a very handy trick known as the **log-derivative trick**. Let's try taking the gradient of $\log p_{\theta}(s)$

$$\nabla_{\theta} \log p_{\theta}(s) = \frac{1}{p_{\theta}(s)} \nabla_{\theta} p_{\theta}(s) \quad \Rightarrow \quad \nabla_{\theta} p_{\theta}(s) = p_{\theta}(s) \nabla_{\theta} \log p_{\theta}(s)$$

(chain rule)

- Plug back in:

$$\sum_s R(s) \nabla_{\theta} p_{\theta}(s) = \sum_s p_{\theta}(s) R(s) \nabla_{\theta} \log p_{\theta}(s)$$

This is an expectation of this

- Now we have put the gradient “inside” the expectation, we can approximate this objective with Monte Carlo samples:

$$\nabla_{\theta} \mathbb{E}_{\hat{s} \sim p_{\theta}(s)} [R(\hat{s})] = \mathbb{E}_{\hat{s} \sim p_{\theta}(s)} [R(\hat{s}) \nabla_{\theta} \log p_{\theta}(\hat{s})] \approx \frac{1}{m} \sum_{i=1}^m R(s_i) \nabla_{\theta} \log p_{\theta}(s_i)$$

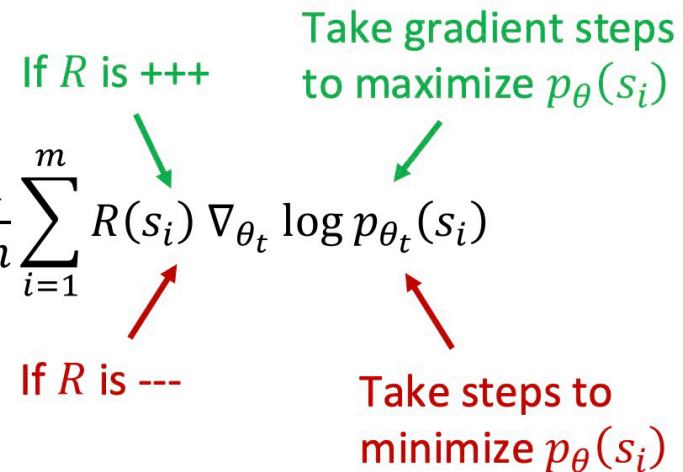
This is why it's called “**reinforcement learning**”: we **reinforce** good actions, increasing the chance they happen again.

- Giving us the update rule: $\theta_{t+1} := \theta_t + \alpha \frac{1}{m} \sum_{i=1}^m R(s_i) \nabla_{\theta_t} \log p_{\theta_t}(s_i)$

This is **heavily simplified**! There is a *lot* more needed to do RL w/ LMs. **Can you see any problems with this objective?**

If R is +++
Take gradient steps to maximize $p_{\theta}(s_i)$

If R is ---
Take steps to minimize $p_{\theta}(s_i)$



- Awesome: now for any **arbitrary, non-differentiable reward function** $R(s)$, we can train our language model to maximize expected reward.
- Not so fast! (Why not?)
- **Problem 1:** human-in-the-loop is expensive!
 - **Solution:** instead of directly asking humans for preferences, **model their preferences** as a separate (NLP) problem! [[Knox and Stone, 2009](#)]

An earthquake hit
San Francisco.
There was minor
property damage,
but no injuries.

$$R(s_1) = 8.0$$


The Bay Area has
good weather but is
prone to
earthquakes and
wildfires.

$$R(s_2) = 1.2$$


Train an LM $RM_\phi(s)$ to
predict human
preferences from an
annotated dataset, then
optimize for RM_ϕ instead.

- **Problem 2:** human judgments are noisy and miscalibrated!
- **Solution:** instead of asking for direct ratings, ask for **pairwise comparisons**, which can be more reliable [[Phelps et al., 2015](#); [Clark et al., 2018](#)]

A 4.2 magnitude
earthquake hit
San Francisco,
resulting in
massive damage.

$$R(s_3) = \begin{matrix} s_3 \\ 4.1? & 6.6? & 3.2? \end{matrix}$$

- **Problem 2:** human judgments are noisy and miscalibrated!
- **Solution:** instead of asking for direct ratings, ask for **pairwise comparisons**, which can be more reliable [[Phelps et al., 2015](#); [Clark et al., 2018](#)]

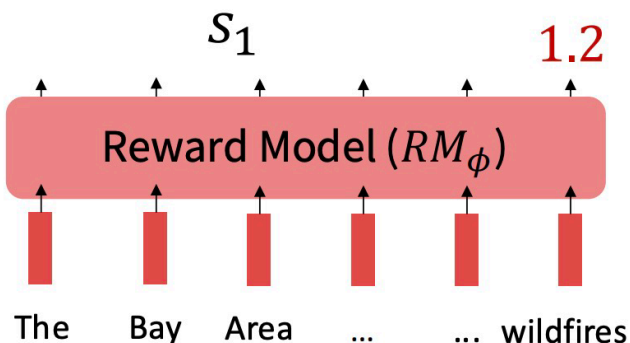
An earthquake hit
San Francisco.
There was minor
property damage,
but no injuries.

>

A 4.2 magnitude
earthquake hit
San Francisco,
resulting in
massive damage.

>

The Bay Area has
good weather but is
prone to
earthquakes and
wildfires.



S_3

Bradley-Terry [1952] paired comparison model

$$J_{RM}(\phi) = -\mathbb{E}_{(s^w, s^l) \sim D} [\log \sigma(RM_\phi(s^w) - RM_\phi(s^l))]$$

“winning”
sample

“losing”
sample

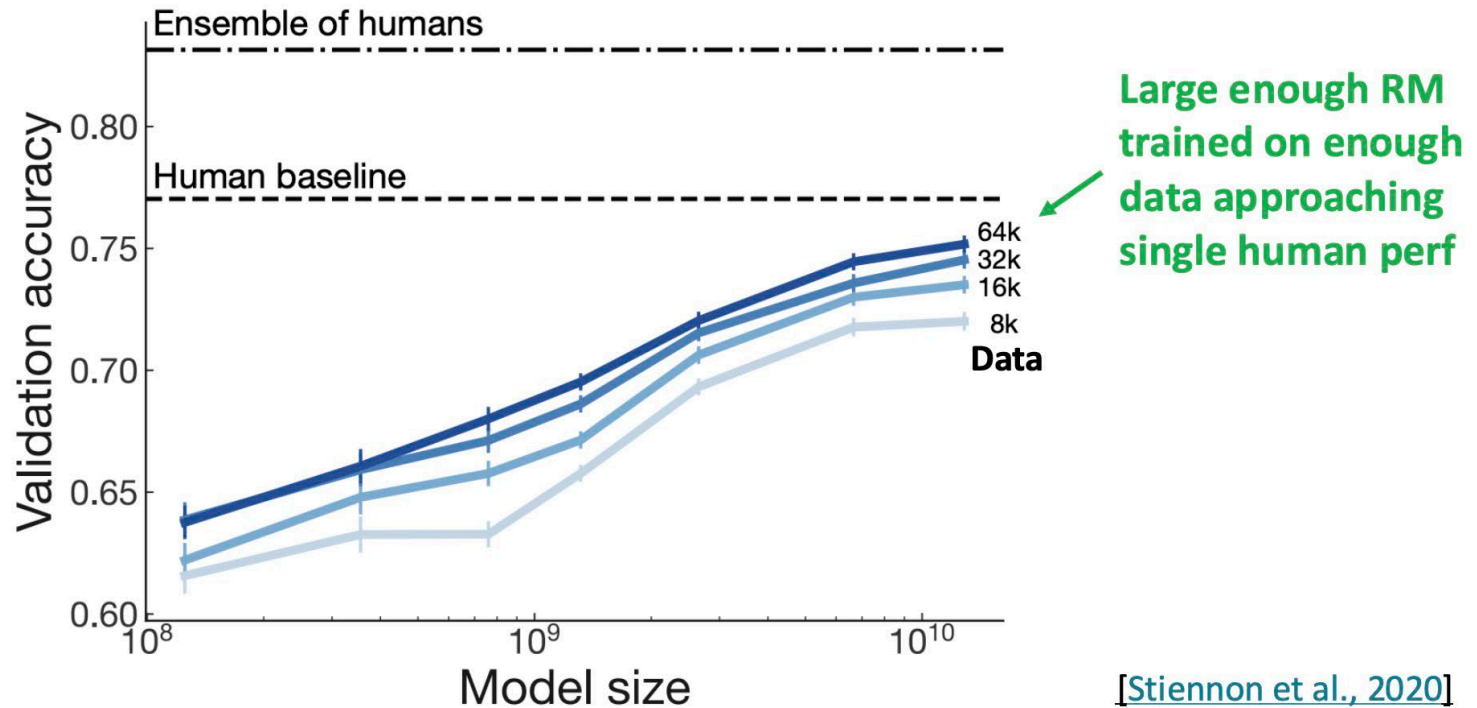
S_2

s^w should score
higher than s^l

Make Your Reward Model Work First!



Evaluate RM on predicting outcome of held-out human judgments

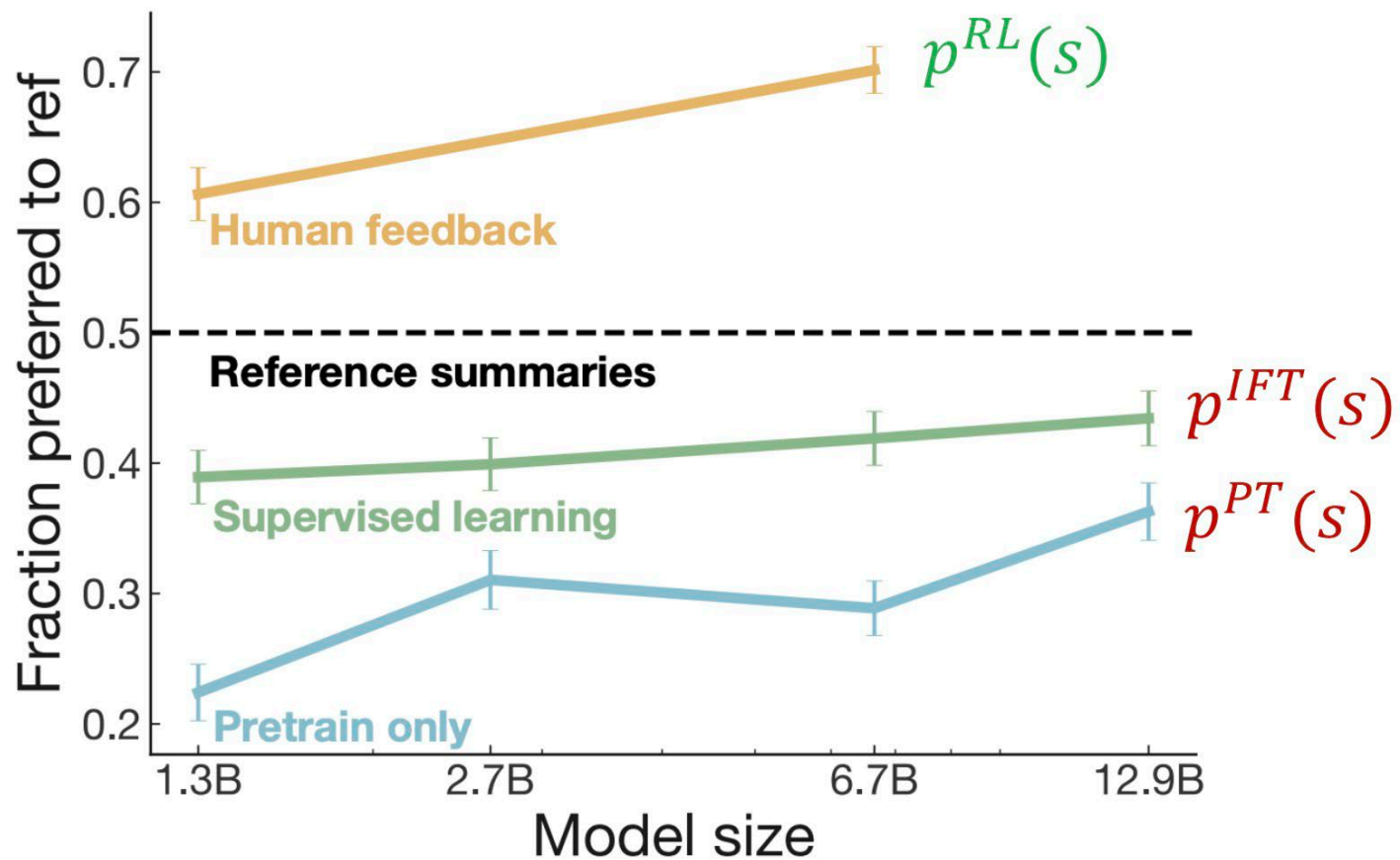


- Finally, we have everything we need:
 - A pretrained (possibly instruction-finetuned) LM $p^{PT}(s)$
 - A reward model $RM_{\phi}(s)$ that produces scalar rewards for LM outputs, trained on a dataset of human comparisons
 - A method for optimizing LM parameters towards an arbitrary reward function.
- Now to do RLHF:
 - Initialize a copy of the model $p_{\theta}^{RL}(s)$, with parameters θ we would like to optimize
 - Optimize the following reward with RL:

$$R(s) = RM_{\phi}(s) - \underbrace{\beta \log \left(\frac{p_{\theta}^{RL}(s)}{p^{PT}(s)} \right)}_{\text{Pay a price when } p_{\theta}^{RL}(s) > p^{PT}(s)}$$

This is a penalty which prevents us from diverging too far from the pretrained model. In expectation, it is known as the **Kullback-Leibler (KL)** divergence between $p_{\theta}^{RL}(s)$ and $p^{PT}(s)$.

Performance of RLHF



30K Tasks

Step 1

Collect demonstration data, and train a supervised policy.

A prompt is sampled from our prompt dataset.

Explain the moon landing to a 6 year old

A labeler demonstrates the desired output behavior.

Some people went to the moon...

This data is used to fine-tune GPT-3 with supervised learning.



Step 2

Collect comparison data, and train a reward model.

A prompt and several model outputs are sampled.

Explain the moon landing to a 6 year old

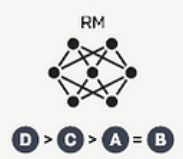
A Explain gravity... B Explain war...

C Moon is natural satellite of... D People went to the moon...

A labeler ranks the outputs from best to worst.

D > C > A = B

This data is used to train our reward model.



Step 3

Optimize a policy against the reward model using reinforcement learning.

A new prompt is sampled from the dataset.

Write a story about frogs

The policy generates an output.



Once upon a time...

The reward model calculates a reward for the output.



The reward is used to update the policy using PPO.

r_k

Tasks collected from labelers:

- **Plain:** We simply ask the labelers to come up with an arbitrary task, while ensuring the tasks had sufficient diversity.
- **Few-shot:** We ask the labelers to come up with an instruction, and multiple query/response pairs for that instruction.
- **User-based:** We had a number of use-cases stated in waitlist applications to the OpenAI API. We asked labelers to come up with prompts corresponding to these use cases.

Use-case	Prompt
Brainstorming	List five ideas for how to regain enthusiasm for my career
Generation	Write a short story where a bear goes to the beach, makes friends with a seal, and then returns home.

PROMPT *Write a short poem about a wise frog.*

COMPLETION GPT-3

Write a short story in which a character has two different names.

Write a short story in which you try to get something back that you have lost.

Write a short story in which a character has a bad dream.

InstructGPT

The frog is a symbol of wisdom
He knows all the secrets of the world
He is a master of disguise
And a great teacher of life
He is a symbol of transformation
And the bringer of change
He is the frog who has seen it all
And knows the meaning of it all



ChatGPT: Optimizing Language Models for Dialogue

Note: OpenAI (and similar companies) are keeping more details secret about ChatGPT training (including data, training parameters, model size)—perhaps to keep a competitive edge...

Methods

We trained this model using Reinforcement Learning from Human Feedback (RLHF), using the same methods as InstructGPT, but with slight differences in the data collection setup. We trained an initial model using supervised fine-tuning: human AI trainers provided conversations in which they played both sides—the user and an AI assistant. We gave the trainers access to model-written suggestions to help them compose their responses. We mixed this new dialogue dataset with the InstructGPT dataset, which we transformed into a dialogue format.

(Instruction finetuning!)



ChatGPT: Optimizing Language Models for Dialogue

Note: OpenAI (and similar companies) are keeping more details secret about ChatGPT training (including data, training parameters, model size)—perhaps to keep a competitive edge...

Methods

To create a reward model for reinforcement learning, we needed to collect comparison data, which consisted of two or more model responses ranked by quality. To collect this data, we took conversations that AI trainers had with the chatbot. We randomly selected a model-written message, sampled several alternative completions, and had AI trainers rank them. Using these reward models, we can fine-tune the model using Proximal Policy Optimization. We performed several iterations of this process.

(RLHF!)